AIM

To implement a custom Ping program using raw ICMP (Internet Control Message Protocol) sockets in C.

Algorithm

Main Program Flow

1. **Create Raw Socket:** Initialize a socket using domain AF_INET, type SOCK_RAW, and protocol IPPROTO_ICMP.
2. **Resolve Host IP:** Convert the domain name or IPv4 string into a valid target IP address using DNS lookup.
3. **Construct ICMP Packet:**
    - Set ICMP type to ICMP_ECHO (Type 8, Code 0).
    - Set Sequence Number and Process Identifier (PID) fields.
    - Calculate and set the Internet Checksum over the entire ICMP header and payload.
4. **Send Echo Request:** Transmit the packet to the destination address using sendto(). Record the transmission timestamp.
5. **Receive Echo Reply:**
    - Wait for incoming packets with recvfrom() within a timeout window.
    - Parse the outer IP header to extract the inner ICMP packet.

    - Verify that the received packet is an ICMP_ECHOREPLY (Type 0) matching the process ID and sequence number.
6. **Calculate Metrics:** Subtract transmission timestamp from receive timestamp to obtain Round Trip Time (RTT).
7. **Display Output:** Print sequence number, TTL (Time to Live), packet size, and RTT. Repeat for desired number of pings.

Code Implementation (ping.c)

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/socket.h>

#include <netinet/in.h>

#include <netinet/ip.h>

#include <netinet/ip_icmp.h>

#include <arpa/inet.h>

#include <netdb.h>

#include <sys/time.h>


#define PACKET_SIZE 64

#define PORT_NO 0

#define TIMEOUT 1 // Timeout in seconds
```

```
// Standard Internet Checksum Algorithm
unsigned short calculate_checksum(unsigned short *buffer, int length) {
    unsigned long sum = 0;
    while (length > 1) {
        sum += *buffer++;
        length -= 2;
    }
    if (length == 1) {
        sum += *(unsigned char *)buffer;
    }
    sum = (sum >> 16) + (sum & 0xFFFF);
    sum += (sum >> 16);
    return (unsigned short)(~sum);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: sudo %s <hostname/IP>\n", argv[0]);
        return 1;
    }

    int sockfd;
```

```
    struct sockaddr_in dest_addr;
    struct hostent *host;
    char packet[PACKET_SIZE];
    struct icmp *icmp_hdr;
    struct timeval timeout, tv_start, tv_end;

    // 1. Resolve Hostname
    host = gethostbyname(argv[1]);
    if (host == NULL) {
        perror("Could not resolve hostname");
        return 1;
    }

    memset(&dest_addr, 0, sizeof(dest_addr));
    dest_addr.sin_family = AF_INET;
    dest_addr.sin_addr = *((struct in_addr *)host-
>h_addr_list[0]);

    // 2. Create Raw Socket for ICMP
    sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
    if (sockfd < 0) {
        perror("Socket creation failed (run as root)");
        return 1;
    }
```

```
    // Set Receive Timeout
    timeout.tv_sec = TIMEOUT;
    timeout.tv_usec = 0;
    setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, (const
char*)&timeout, sizeof(timeout));


    printf("PING %s (%s) %d bytes of data.\n", argv[1],
inet_ntoa(dest_addr.sin_addr), PACKET_SIZE);


    for (int seq = 1; seq <= 4; seq++) {
        memset(packet, 0, PACKET_SIZE);


        // 3. Construct ICMP Header
        icmp_hdr = (struct icmp *)packet;
        icmp_hdr->icmp_type = ICMP_ECHO;
        icmp_hdr->icmp_code = 0;
        icmp_hdr->icmp_id = getpid();
        icmp_hdr->icmp_seq = seq;
        icmp_hdr->icmp_cksum = 0;


        // Fill remaining bytes with sample data payload
        for (int i = sizeof(struct icmp); i < PACKET_SIZE; i++) {
            packet[i] = i + '0';
```

```
        }

        icmp_hdr->icmp_cksum = calculate_checksum((unsigned
short *)packet, PACKET_SIZE);

        // 4. Send Packet & Record Time
        gettimeofday(&tv_start, NULL);
        if (sendto(sockfd, packet, PACKET_SIZE, 0, (struct
sockaddr *)&dest_addr, sizeof(dest_addr)) <= 0) {
            perror("Failed to send packet");
            continue;
        }

        // 5. Receive Reply
        char recv_buf[1024];
        struct sockaddr_in from_addr;
        socklen_t from_len = sizeof(from_addr);

        int bytes_rcvd = recvfrom(sockfd, recv_buf,
sizeof(recv_buf), 0, (struct sockaddr *)&from_addr, &from_len);
        gettimeofday(&tv_end, NULL);

        if (bytes_rcvd < 0) {
            printf("Request timed out for icmp_seq %d\n", seq);
        } else {
```

```
            // Extract IP header to inspect TTL and payload
            struct ip *ip_hdr = (struct ip *)recv_buf;
            int ip_hdr_len = ip_hdr->ip_hl * 4;
            struct icmp *reply_icmp = (struct icmp *)(recv_buf +
ip_hdr_len);

            // Calculate RTT in milliseconds
            double rtt = (tv_end.tv_sec - tv_start.tv_sec) * 1000.0 +
                         (tv_end.tv_usec - tv_start.tv_usec) / 1000.0;

            if (reply_icmp->icmp_type == ICMP_ECHOREPLY) {
                printf("%d bytes from %s: icmp_seq=%d ttl=%d
time=%.2f ms\n",
                       bytes_rcvd - ip_hdr_len,
                       inet_ntoa(from_addr.sin_addr),
                       reply_icmp->icmp_seq,
                       ip_hdr->ip_ttl,
                       rtt);
            }
        }
        sleep(1); // Wait 1 second between requests
    }

    close(sockfd);
```

```
    return 0;
}
```

Input / Output

Input

Command Terminal:

sudo ./myping 127.0.0.1

Output

Server Terminal:

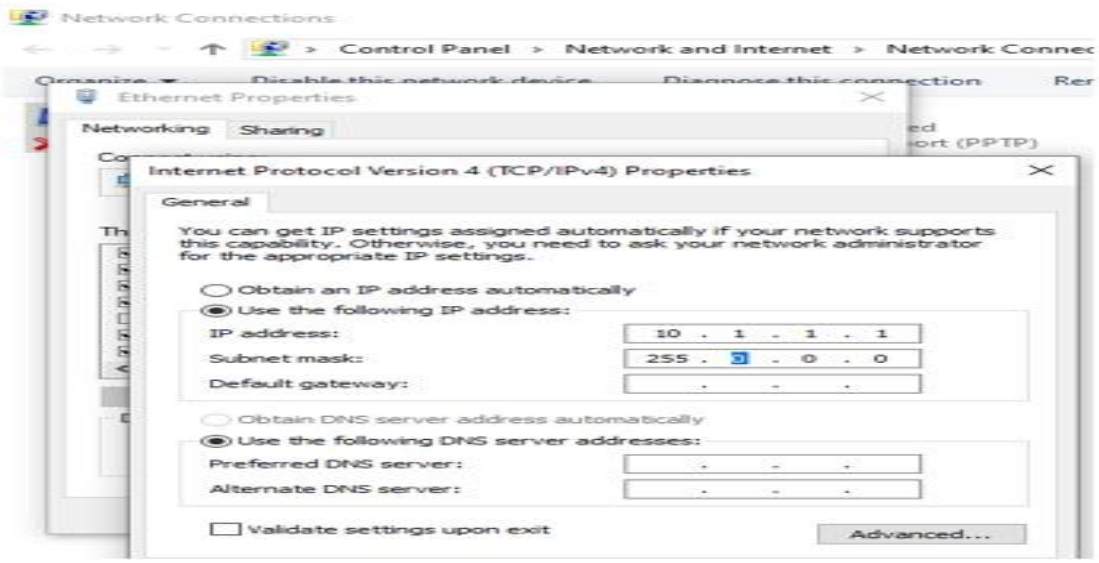
PING 127.0.0.1 (127.0.0.1) 64 bytes of data.

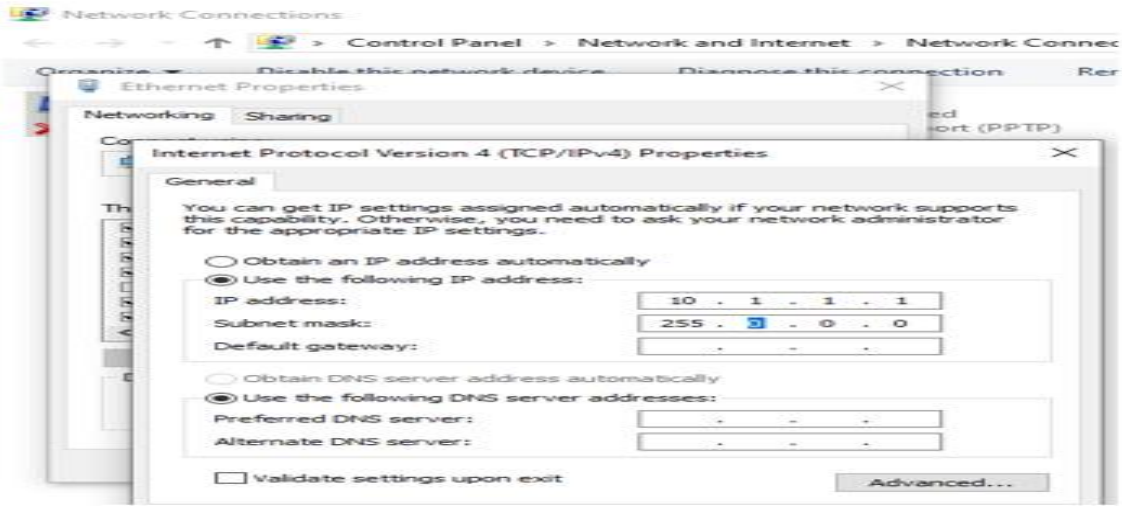
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.08 ms

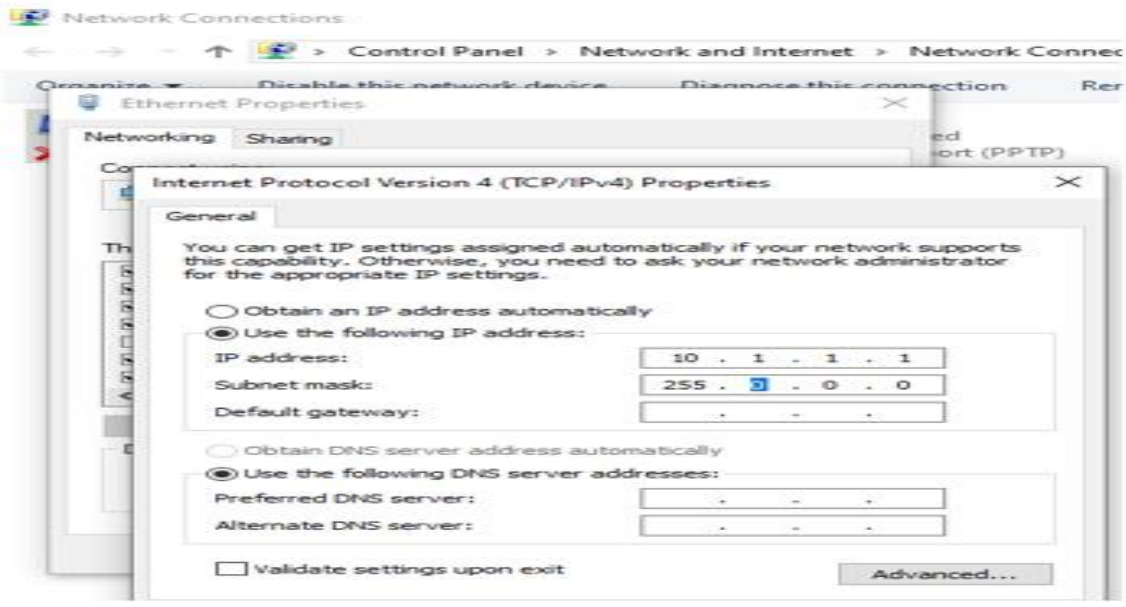
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.06 ms

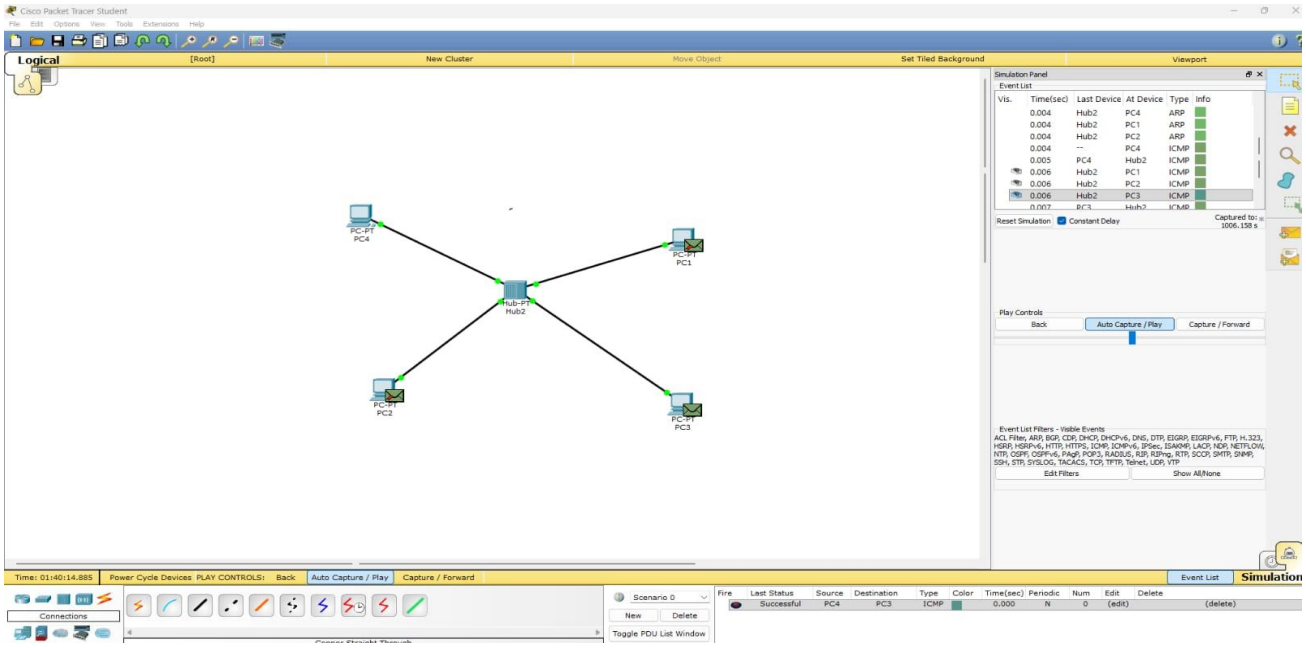
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.07 ms

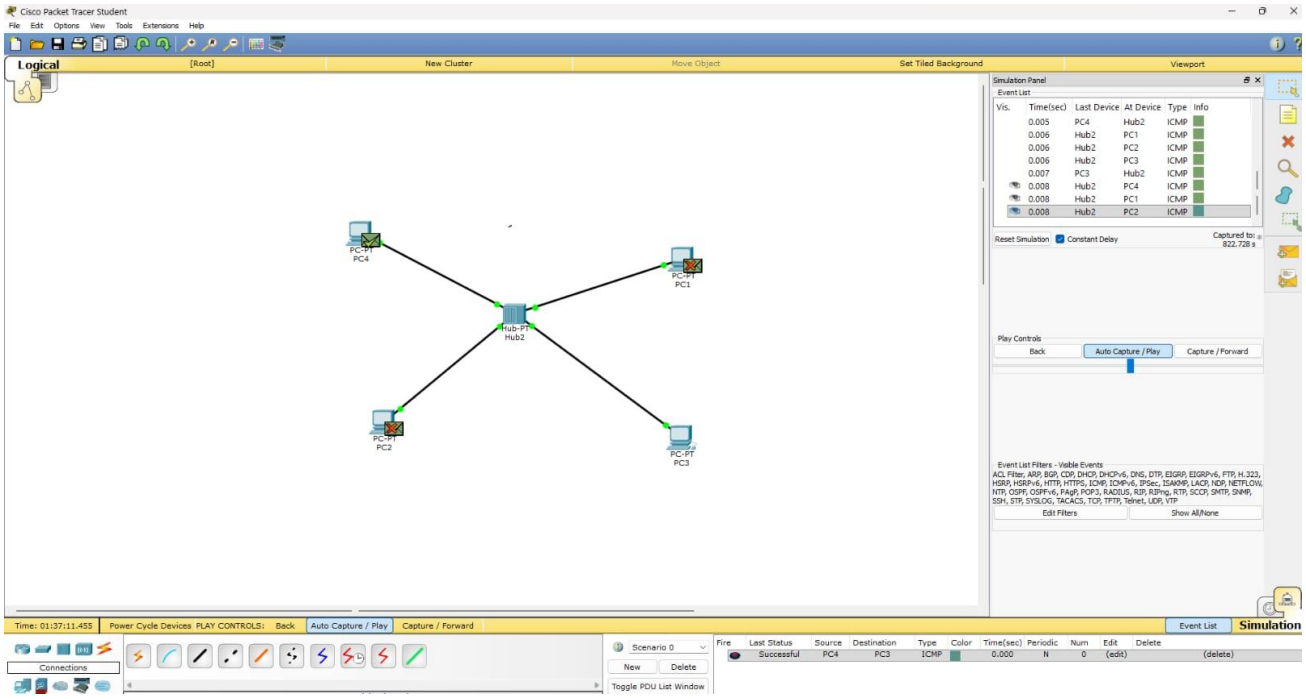
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.05 ms

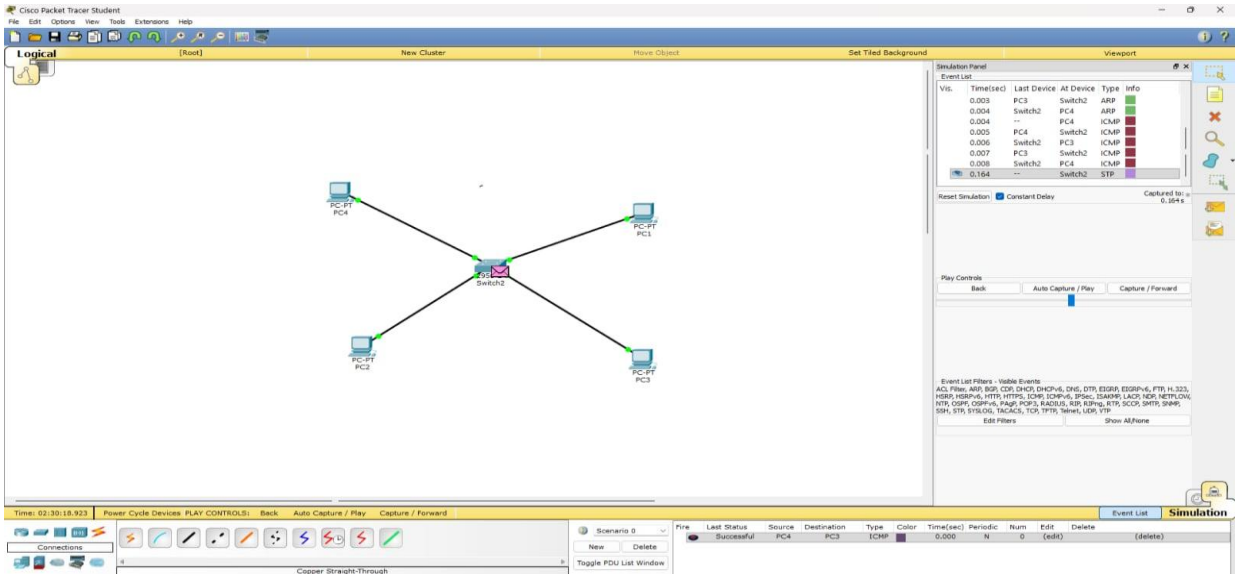

Network Connections
Control Panel › Network and Internet › Network Connec
Ethernet Properties
Networking
Sharing
Internet Protocol Version 4 (TCP/IPv4) Properties
General
You can get IP settings assigned automatically if your network supports this capability. Otherwise, you need to ask your network administrator for the appropriate IP settings.
Obtain an IP address automatically
Use the following IP address:
IP address: 10 . 1 . 1 . 1
Subnet mask: 255 . 0 . 0 . 0
Default gateway:
Obtain DNS server address automatically
Use the following DNS server addresses:
Preferred DNS server:
Alternate DNS server:
Validate settings upon exit
Advanced...

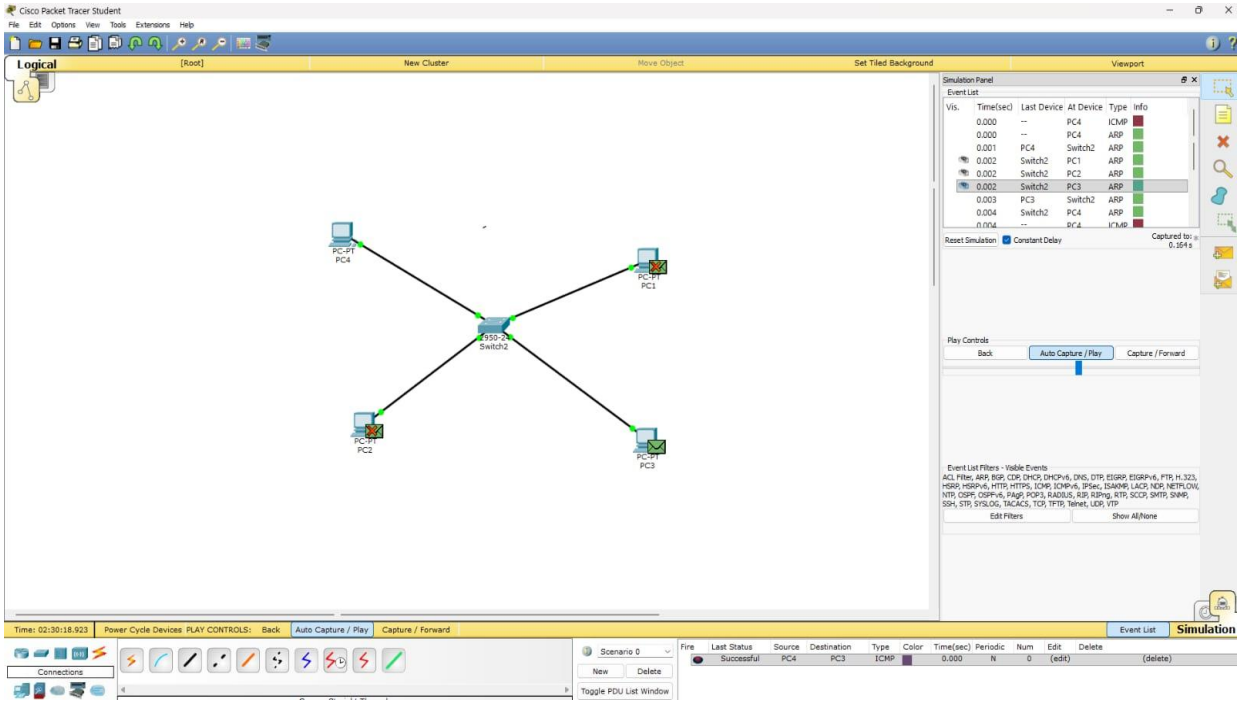

Network Connections
Control Panel › Network and Internet › Network Connec
Ethernet Properties
Networking
Sharing
Internet Protocol Version 4 (TCP/IPv4) Properties
General
You can get IP settings assigned automatically if your network supports this capability. Otherwise, you need to ask your network administrator for the appropriate IP settings.
Obtain an IP address automatically
Use the following IP address:
IP address: 10 . 1 . 1 . 1
Subnet mask: 255 . 0 . 0 . 0
Default gateway:
Obtain DNS server address automatically
Use the following DNS server addresses:
Preferred DNS server:
Alternate DNS server:
Validate settings upon exit
Advanced...

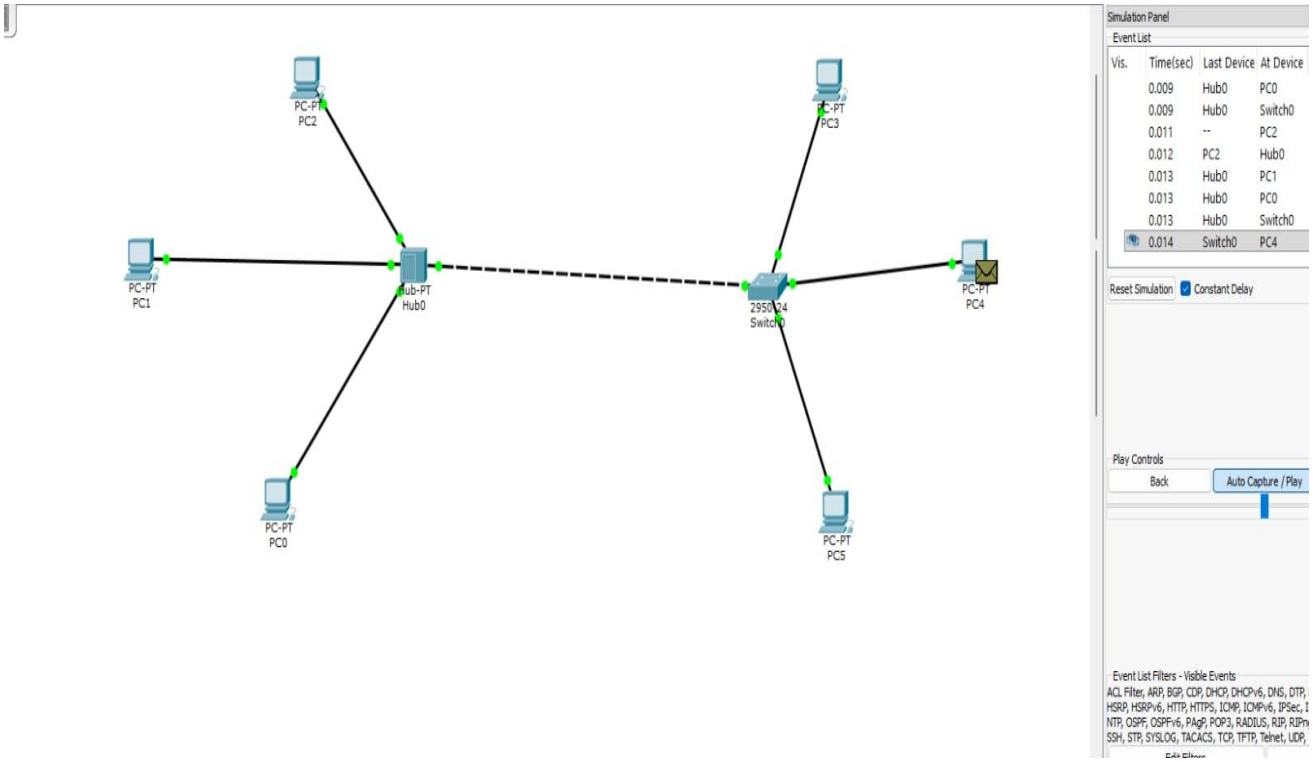

Network Connections
Control Panel › Network and Internet › Network Connec
Ethernet Properties
Networking
Sharing
Internet Protocol Version 4 (TCP/IPv4) Properties
General
You can get IP settings assigned automatically if your network supports this capability. Otherwise, you need to ask your network administrator for the appropriate IP settings.
Obtain an IP address automatically
Use the following IP address:
IP address: 10 . 1 . 1 . 1
Subnet mask: 255 . 0 . 0 . 0
Default gateway:
Obtain DNS server address automatically
Use the following DNS server addresses:
Preferred DNS server:
Alternate DNS server:
Validate settings upon exit
Advanced...


Cisco Packet Tracer Student
Logical
[Root]
New Cluster
Move Object
Set Tiled Background
Viewport
Hub-PT
Hub2
PC-PT
PC4
PC1
PC2
PC3
Simulation Panel
Event List
Vis. Time(sec) Last Device At Device Type Info
0.004 Hub2 PC4 ARP
0.004 Hub2 PC1 ARP
0.004 Hub2 PC2 ARP
0.004 --- PC4 ICMP
0.005 PC4 Hub2 ICMP
0.006 Hub2 PC1 ICMP
0.006 Hub2 PC2 ICMP
0.006 Hub2 PC3 ICMP
Reset Simulation
Constant Delay
Play Controls
Back
Auto Capture / Play
Capture / Forward
Edit Filters
Show All/None
Time: 01:40:14.885
Power Cycle Devices
PLAY CONTROLS:
Back
Auto Capture / Play
Capture / Forward
Event List
Simulation
Connections
Scenario 0
New
Delete
Toggle PDU List Window
Fire Last Status Source Destination Type Color Time(sec) Periodic Num Edit Delete
Successful PC4 PC3 ICMP 0.000 N 0 (edit) (delete)


Cisco Packet Tracer Student
Logical
[Root]
New Cluster
Move Object
Set Tiled Background
Viewport
Hub-PT
Hub2
PC-PT
PC4
PC1
PC2
PC3
Simulation Panel
Event List
Vis. Time(sec) Last Device At Device Type Info
0.005 PC4 Hub2 ICMP
0.006 Hub2 PC1 ICMP
0.006 Hub2 PC2 ICMP
0.006 Hub2 PC3 ICMP
0.007 PC3 Hub2 ICMP
0.008 Hub2 PC4 ICMP
0.008 Hub2 PC1 ICMP
0.008 Hub2 PC2 ICMP
Reset Simulation
Constant Delay
Play Controls
Back
Auto Capture / Play
Capture / Forward
Edit Filters
Show All/None
Time: 01:37:11.455
Power Cycle Devices
PLAY CONTROLS:
Back
Auto Capture / Play
Capture / Forward
Event List
Simulation
Connections
Scenario 0
New
Delete
Toggle PDU List Window
Fire Last Status Source Destination Type Color Time(sec) Periodic Num Edit Delete
Successful PC4 PC3 ICMP 0.000 N 0 (edit) (delete)


Cisco Packet Tracer Student
Logical
PC-PT PC4
PC-PT PC1
Switch2
PC-PT PC2
PC-PT PC3
Simulation Panel
Event List
Time(sec) Last Device At Device Type Info
0.003 PC3 Switch2 ARP
0.004 Switch2 PC4 ARP
0.004 -- PC4 ICMP
0.005 PC4 Switch2 ICMP
0.006 Switch2 PC3 ICMP
0.007 PC3 Switch2 ICMP
0.008 Switch2 PC4 ICMP
0.164 -- Switch2 STP
Reset Simulation
Constant Delay
Play Controls
Back
Auto Capture / Play
Capture / Forward
Edit Filters
Show All/None
Time: 02:30:18.923
Power Cycle Devices
PLAY CONTROLS:
Event List
Simulation
Connections
Copper Straight-Through
Scenario 0
New
Delete
Toggle PDU List Window
Fire Last Status Source Destination Type Color Time(sec) Periodic Num Edit Delete
Successful PC4 PC3 ICMP 0.000 N 0 (edit) (delete)


Cisco Packet Tracer Student
Logical
PC-PT PC4
PC-PT PC1
2950-24 Switch2
PC-PT PC2
PC-PT PC3
Simulation Panel
Event List
Time(sec) Last Device At Device Type Info
0.000 -- PC4 ICMP
0.000 -- PC4 ARP
0.001 PC4 Switch2 ARP
0.002 Switch2 PC1 ARP
0.002 Switch2 PC2 ARP
0.002 Switch2 PC3 ARP
0.003 PC3 Switch2 ARP
0.004 Switch2 PC4 ARP
Reset Simulation
Constant Delay
Play Controls
Back
Auto Capture / Play
Capture / Forward
Edit Filters
Show All/None
Time: 02:30:18.923
Power Cycle Devices
PLAY CONTROLS:
Event List
Simulation
Connections
Scenario 0
New
Delete
Toggle PDU List Window
Fire Last Status Source Destination Type Color Time(sec) Periodic Num Edit Delete
Successful PC4 PC3 ICMP 0.000 N 0 (edit) (delete)


PC-PT
PC2
PC-PT
PC3
PC-PT
PC1
Hub-PT
Hub0
2950-24
Switch0
PC-PT
PC4
PC-PT
PC0
PC-PT
PC5
Simulation Panel
Event List
Vis.
Time(sec)
Last Device
At Device
0.009
Hub0
PC0
0.009
Hub0
Switch0
0.011
--
PC2
0.012
PC2
Hub0
0.013
Hub0
PC1
0.013
Hub0
PC0
0.013
Hub0
Switch0
0.014
Switch0
PC4
Reset Simulation
Constant Delay
Play Controls
Back
Auto Capture / Play
Event List Filters - Visible Events
ACL Filter, ARP, BGP, CDP, DHCP, DHCPv6, DNS, DTP,
HSRP, HSRPv6, HTTP, HTTPS, ICMP, ICMPv6, IPSec,
NTP, OSPF, OSPFv6, PAgP, POP3, RADIUS, RIP, RIPn
SSH, STP, SYSLOG, TACACS, TCP, TFTP, Telnet, UDP,

EXP 15

AIM

To analyze different types of web logs using the Webalizer log analysis tool and generate web traffic reports.

Procedure

Step 1: Run Webalizer

1. Download and extract the Webalizer executable (webalizer.exe) on Windows.
2. Place the executable in a dedicated working folder (e.g., C:\webalizer).

Step 2: Input Web Log File

1. Obtain or download a sample web server access log file (e.g., access.log from Apache, Nginx, or IIS).
2. Move the log file into the same directory as webalizer.exe or update the webalizer.conf configuration file to point to the input log location:

LogFile access.log

OutputDir ./output

Step 3: Run Webalizer

1. Open Command Prompt (cmd) and navigate to the Webalizer directory:

DOS

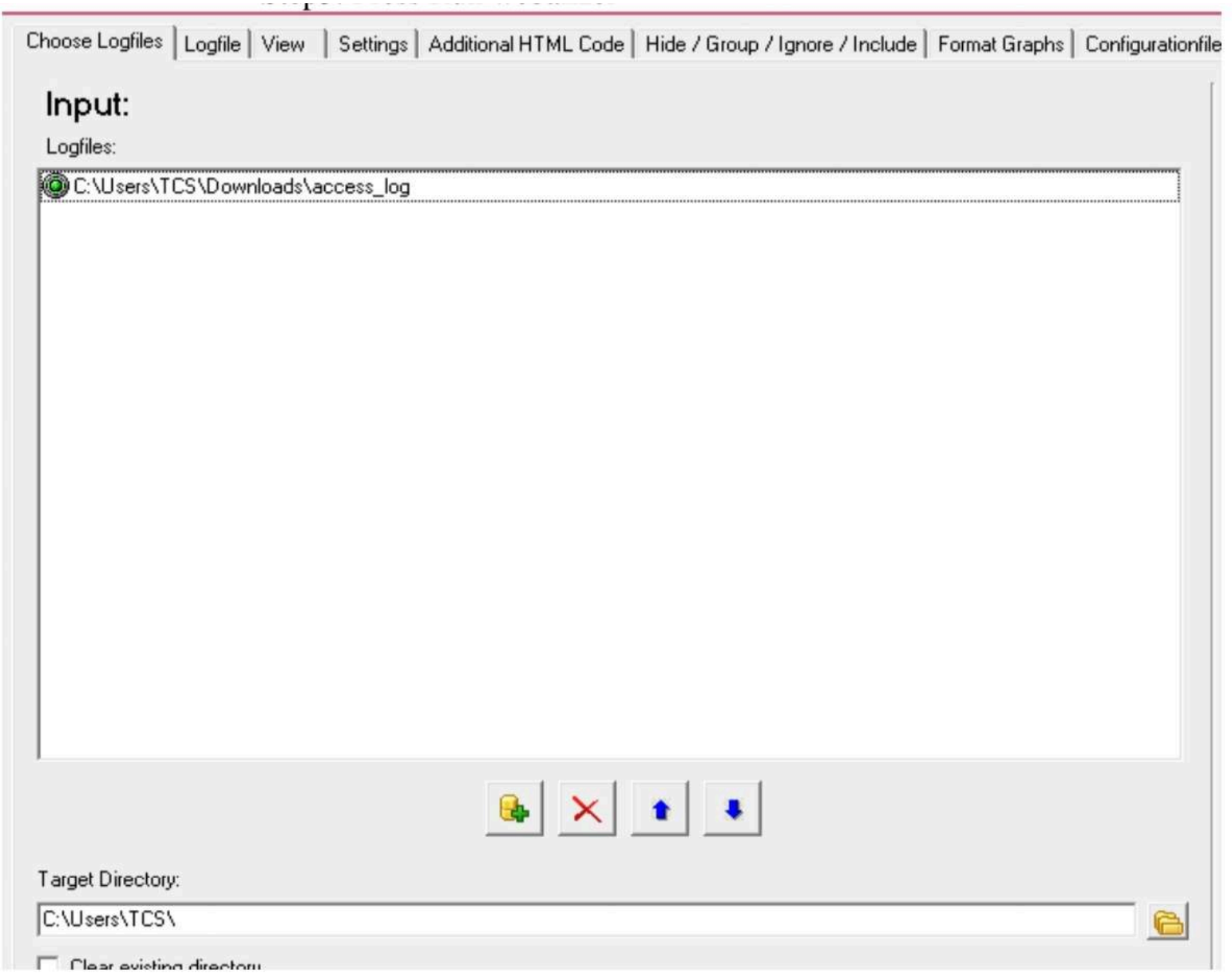
cd C:\webalizer

2. Execute Webalizer specifying the log file:

DOS

webalizer.exe access.log

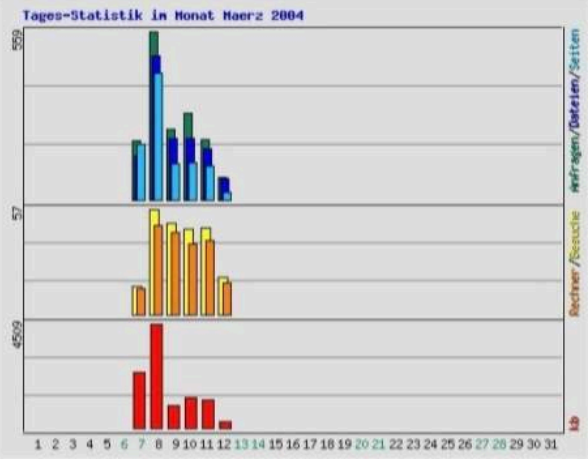
3. Open the generated index.html file in any web browser to view the generated traffic report.


Choose Logfiles
Logfile
View
Settings
Additional HTML Code
Hide / Group / Ignore / Include
Format Graphs
Configurationfile
Input:
Logfiles:
C:\Users\TCS\Downloads\access_log
Target Directory:
C:\Users\TCS\

Monthly statistics

March 2004 | Last 12 Months | Monthly Statistics | Daily Statistics | Hourly Statistics | URLs

Entry Pages | Exit Pages | Errors | Hosts | Referrer | Searches | Usernames | Useragents | Countries



Seiten - Dokument-Abfragen (z.B. html, asp, txt); Dateien - erfolgreiche Anfragen für alle Typen (z.B. html, gif, css); Treffer - alle Anfragen, inklusive Fehler;

Tages-Statistik im Monat Maerz 2004

Tag	Anfragen			Dateien			Seiten			Besuche			Rechner			kb		
	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum
7	197 12.74%	25	29	147 11.54%	18	21	186 18.94%	23	27	15 6.41%	2	3	14 8.19%	2	3	2383 22.27%	298	542
8	559 36.16%	24	63	478 37.52%	21	56	419 42.67%	18	40	57 24.36%	2	7	48 28.07%	2	7	4509 42.14%	196	386
9	232 15.01%	11	29	205 16.09%	10	29	118 12.02%	6	16	49 20.94%	2	5	44 25.73%	2	5	955 8.93%	45	138
10	286 18.50%	14	102	205 16.09%	10	34	124 12.63%	6	28	46 19.66%	2	5	38 22.22%	2	4	1336 12.49%	67	379

Hosts

March 2004 | Last 12 Months | Monthly Statistics | Daily Statistics | Hourly Statistics | URLs

Entry Pages | Exit Pages | Errors | Hosts | Referrer | Searches | Usernames | Useragents | Countries

Top 20 von 171 Rechnern (IP-Adressen)

#	Anfragen	Dateien	Seiten	kb	Besuche	Dauer	Land	Rechnername
1	100 6.47%	83 6.51%	46 4.68%	472 4.41%	10 4.27%	8.48	39.85	shawcable.net
2	72 4.66%	71 5.57%	52 5.30%	86 0.80%	71 30.34%	0.09	6.62	inktomisearch.com
3	47 3.04%	41 3.22%	43 4.38%	613 5.73%	4 1.71%	3.77	15.10	overture.com
4	44 2.85%	42 3.30%	23 2.34%	244 2.28%	2 0.85%	4.51	4.70	hevanet.com
5	35 2.26%	29 2.28%	14 1.43%	218 2.03%	7 2.99%	1.31	6.00	bc.ca
6	29 1.88%	28 2.20%	14 1.43%	97 0.91%	1 0.43%	3.43	3.43	panduit.com
7	23 1.49%	14 1.10%	10 1.02%	135 1.26%	1 0.43%	12.35	12.35	geovariances.fr
8	22 1.42%	22 1.73%	8 0.81%	67 0.63%	1 0.43%	3.72	3.72	cox.net
9	19 1.23%	19 1.49%	7 0.71%	61 0.57%	1 0.43%	1.52	1.52	ac.il
10	19 1.23%	11 0.86%	10 1.02%	51 0.48%	2 0.85%	3.04	5.87	netinfo.bg
11	15 0.97%	14 1.10%	13 1.32%	130 1.22%	3 1.28%	1.29	3.87	e-i.net
12	13 0.84%	13 1.02%	13 1.32%	120 1.12%	1 0.43%	3.88	3.88	telia.net
13	13 0.84%	13 1.02%	1 0.10%	28 0.26%	1 0.43%	0.13	0.13	net.ar
14	13 0.84%	13 1.02%	2 0.20%	30 0.28%	1 0.43%	0.87	0.87	dhl.com
15	12 0.78%	11 0.86%	9 0.92%	68 0.63%	1 0.43%	25.53	25.53	tiscali.de
16	12 0.78%	12 0.94%	1 0.10%	27 0.25%	1 0.43%	0.33	0.33	wtbts.org
17	12 0.78%	12 0.94%	1 0.10%	27 0.25%	1 0.43%	0.10	0.10	qwest.net
18	11 0.71%	11 0.86%	7 0.71%	41 0.38%	2 0.85%	0.56	1.03	radiant.net
19	10 0.65%	9 0.71%	7 0.71%	41 0.38%	1 0.43%	1.40	1.40	net.au
20	10 0.65%	10 0.78%	2 0.20%	61 0.57%	1 0.43%	0.17	0.17	3_343_lt_someone

Top 10 von 171 Rechnern (IP-Adressen) sortiert nach kb

#	Anfragen	Dateien	Seiten	kb	Besuche	Dauer	Land	Rechnername
1	47 3.04%	41 3.22%	43 4.38%	613 5.73%	4 1.71%	3.77	15.10	overture.com
2	100 6.47%	83 6.51%	46 4.68%	472 4.41%	10 4.27%	8.48	39.85	shawcable.net

User-agents

March 2004 | Last 12 Months | Monthly Statistics | Daily Statistics | Hourly Statistics | URLs

Entry Pages | Exit Pages | Errors | Hosts | Referrer | Searches | Usernames | Useragents | Countries

9	72	4.66%	71	5.57%	52	5.30%	86	0.80%	71	30.34%	0.09	6.62	**inktomisearch.com**
10	4	0.26%	4	0.31%	4	0.41%	74	0.69%	4	1.71%	0.00	0.00	**rhyolite.com**

Top 1 von 1 Verweise

#	Anfragen		Besuche		Verweise
1	1546	100.00%	234	100.00%	- (Direkte Anfrage)

Top 1 von 1 Anwenderprogramme

#	Anfragen		kb		Besuche		Anwenderprogramm
1	1546	100.00%	10699	100.00%	234	100.00%	-



Top 1 von 1 Ländern

#	Anfragen		Dateien		kb		Besuche		Land
1	1543	99.81%	1271	99.76%	10684	99.85%	231	100.00%	Unbekannte Adressen

100% Micro$oft free!

EXP 14

AIM

To write and implement a network packet sniffer using Raw Sockets in C to capture and parse IP, TCP, and UDP network headers.

Algorithm

Main Program Flow

1. **Create Raw Socket:** Open a socket using domain AF_INET, type SOCK_RAW, and protocol IPPROTO_TCP (or ETH_P_ALL for lower-level link layer capture).
2. **Buffer Allocation:** Allocate a buffer (e.g., 65536 bytes) to hold incoming network frames and packets.
3. **Capture Loop:**
    - Call recvfrom() to continuously capture network packets passing through the interface.
4. **Parse Headers:**
    - **Extract IP Header:** Cast the buffer pointer to an struct iphdr pointer. Extract fields like Version, Header Length, TTL, Protocol, Source IP, and Destination IP.
    - **Identify Protocol:** Check the protocol field (ip_hdr->protocol).
    - **Extract Transport Header:**
        - If Protocol is 6 (TCP): Cast buffer offset to struct tcphdr and extract Source Port, Destination Port, and Flags (SYN, ACK, FIN, etc.).

      If Protocol is 17 (UDP): Cast buffer offset to struct udphdr and extract Source Port, Destination Port, and Length.
5. **Display Packet Info:** Print formatted packet metadata to the console.
6. **Clean Up:** Close the socket upon receiving a termination signal.

Code Implementation (packet_sniffer.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netinet/udp.h>
#include <arpa/inet.h>

#define BUFFER_SIZE 65536

void process_packet(unsigned char *buffer, int size) {
    // Extract IP Header
    struct iphdr *ip_hdr = (struct iphdr *)buffer;
    unsigned int ip_hdr_len = ip_hdr->ihl * 4;

    struct sockaddr_in source, dest;
    memset(&source, 0, sizeof(source));
    source.sin_addr.s_addr = ip_hdr->saddr;

    memset(&dest, 0, sizeof(dest));
    dest.sin_addr.s_addr = ip_hdr->daddr;

    printf("\n================ Packet Captured
================\n");
```

```
printf("IP Header:\n");
printf("   |-IP Version        : %d\n", (unsigned int)ip_hdr->version);
printf("   |-Header Length     : %d Bytes\n", ip_hdr_len);
printf("   |-Time To Live (TTL): %d\n", ip_hdr->ttl);
printf("   |-Protocol          : %d\n", ip_hdr->protocol);
printf("   |-Source IP         : %s\n", inet_ntoa(source.sin_addr));
printf("   |-Destination IP    : %s\n", inet_ntoa(dest.sin_addr));

// Handle TCP Packets
if (ip_hdr->protocol == IPPROTO_TCP) {
    struct tcphdr *tcp_hdr = (struct tcphdr *)(buffer + ip_hdr_len);
    printf("TCP Header:\n");
    printf("   |-Source Port       : %u\n", ntohs(tcp_hdr->source));
    printf("   |-Destination Port  : %u\n", ntohs(tcp_hdr->dest));
    printf("   |-Sequence Number   : %u\n", ntohl(tcp_hdr->seq));
    printf("   |-Acknowledge No    : %u\n", ntohl(tcp_hdr->ack_seq));
    printf("   |-SYN Flag          : %d\n", tcp_hdr->syn);
    printf("   |-ACK Flag          : %d\n", tcp_hdr->ack);
    printf("   |-FIN Flag          : %d\n", tcp_hdr->fin);
}
// Handle UDP Packets
else if (ip_hdr->protocol == IPPROTO_UDP) {
    struct udphdr *udp_hdr = (struct udphdr *)(buffer + ip_hdr_len);
    printf("UDP Header:\n");
    printf("   |-Source Port       : %u\n", ntohs(udp_hdr->source));
    printf("   |-Destination Port  : %u\n", ntohs(udp_hdr->dest));
```

```
        printf("   |-UDP Length        : %u\n", ntohs(udp_hdr->len));
    }

printf("============================================
=\n");
}

int main() {
    int raw_sock;
    socklen_t saddr_len;
    struct sockaddr saddr;
    unsigned char *buffer = (unsigned char *)malloc(BUFFER_SIZE);

    if (buffer == NULL) {
        printf("Buffer allocation failed.\n");
        return 1;
    }

    // Create raw socket to capture IP packets
    raw_sock = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
    if (raw_sock < 0) {
        perror("Socket creation error (ensure you run with sudo)");
        free(buffer);
        return 1;
    }

    printf("Starting Packet Sniffer...\nPress Ctrl+C to stop.\n");
```

```
    // Capture 3 packets as an example output
    int packet_count = 0;
    while (packet_count < 3) {
        saddr_len = sizeof(saddr);
        int data_size = recvfrom(raw_sock, buffer, BUFFER_SIZE, 0,
&saddr, &saddr_len);

        if (data_size < 0) {
            perror("Recvfrom error");
            free(buffer);
            close(raw_sock);
            return 1;
        }

        process_packet(buffer, data_size);
        packet_count++;
    }

    close(raw_sock);
    free(buffer);
    return 0;
}
```

Input / Output

Input

Command Terminal:

sudo ./sniffer

Output

Terminal Screen:

Starting Packet Sniffer...

Press Ctrl+C to stop.

================ Packet Captured ================

IP Header:

|-IP Version : 4

|-Header Length : 20 Bytes

|-Time To Live (TTL): 64

|-Protocol : 6

|-Source IP : 192.168.1.5

|-Destination IP : 93.184.216.34

TCP Header:

|-Source Port : 49152

|-Destination Port : 80

|-Sequence Number : 10423011

|-Acknowledge No : 0

|-SYN Flag : 1

|-ACK Flag : 0

|-FIN Flag : 0

==

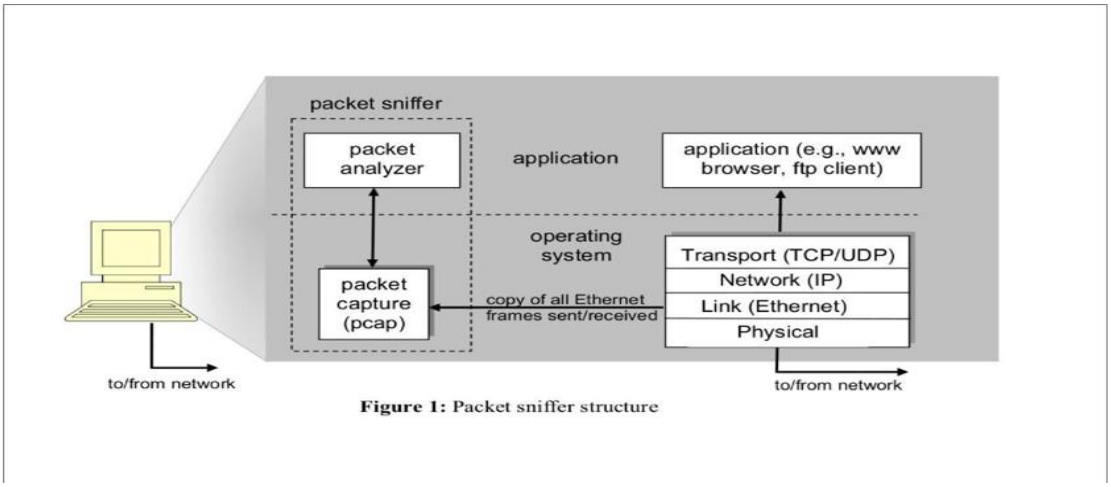
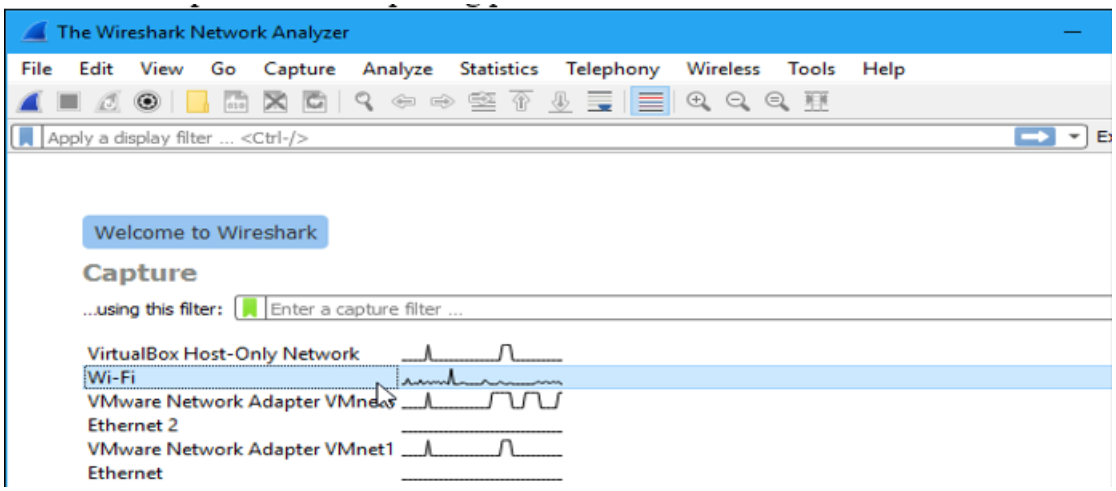
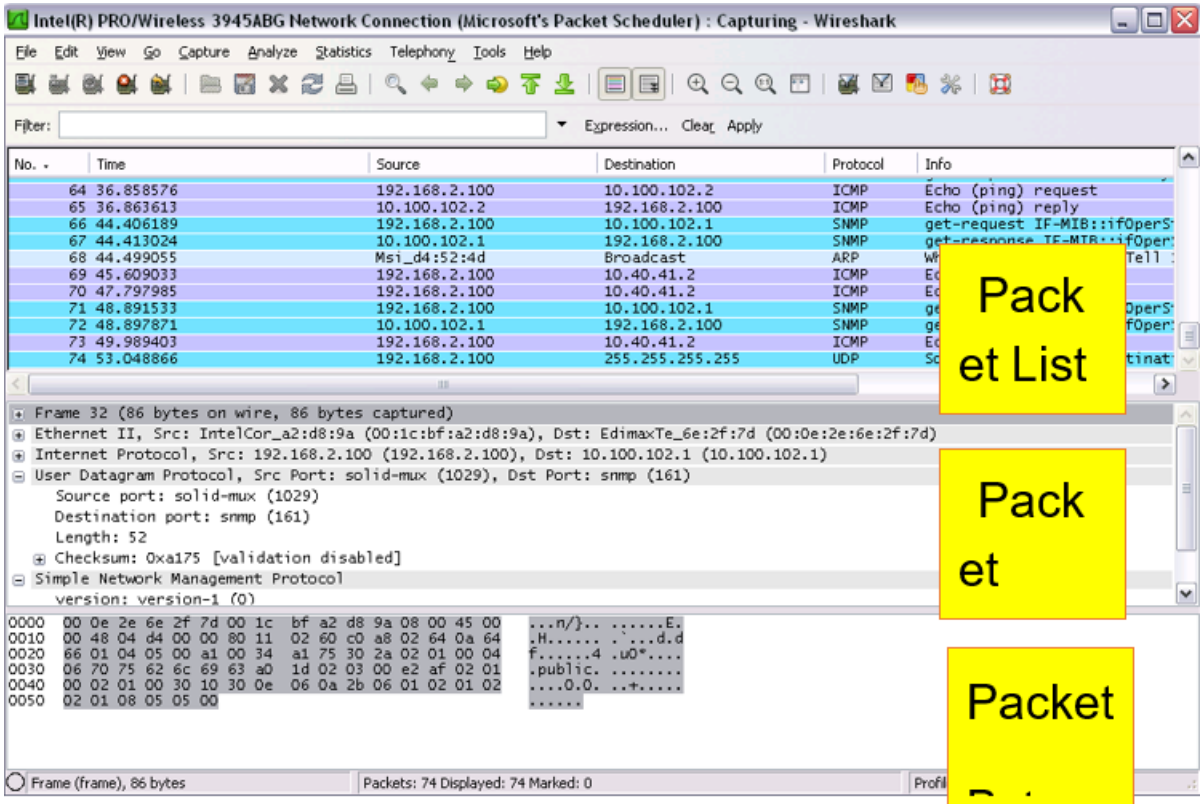
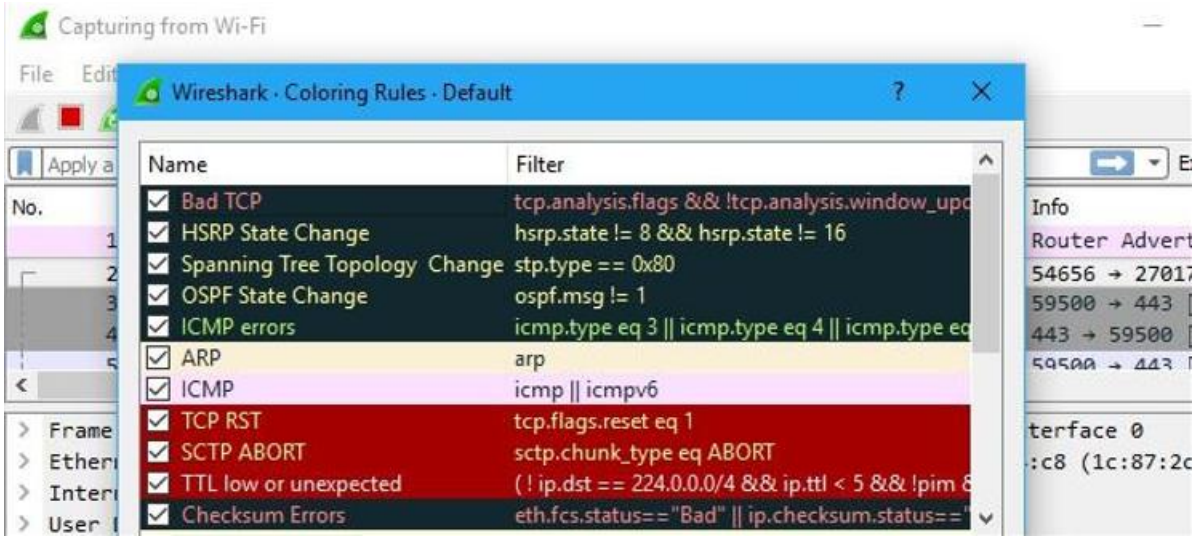
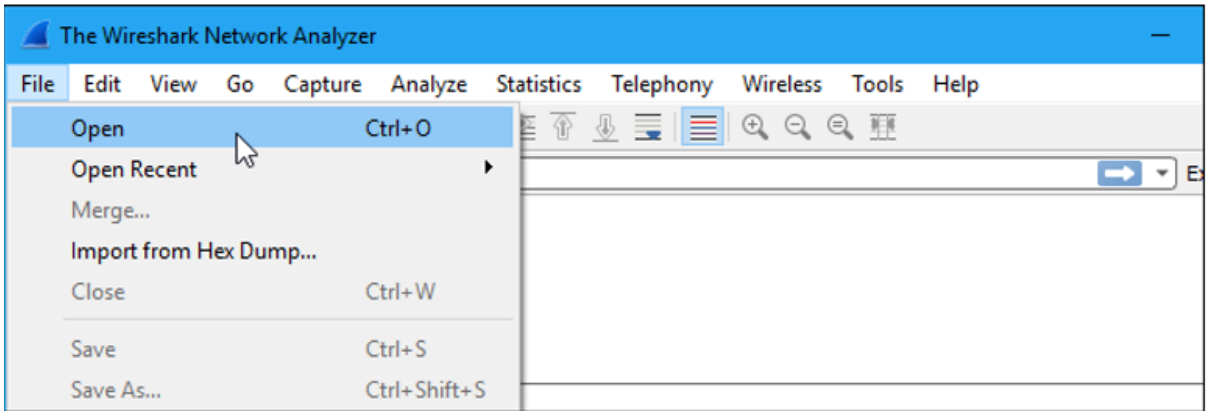


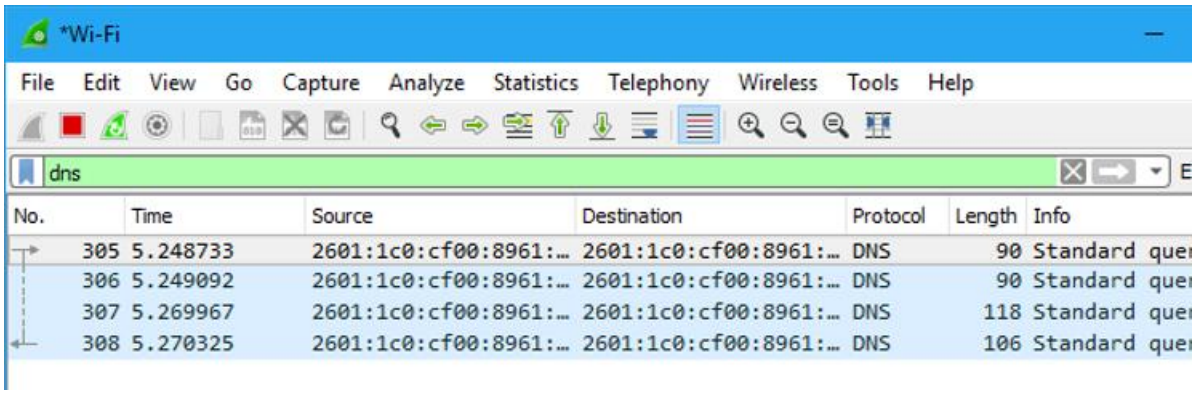
Figure 1: Packet sniffer structure

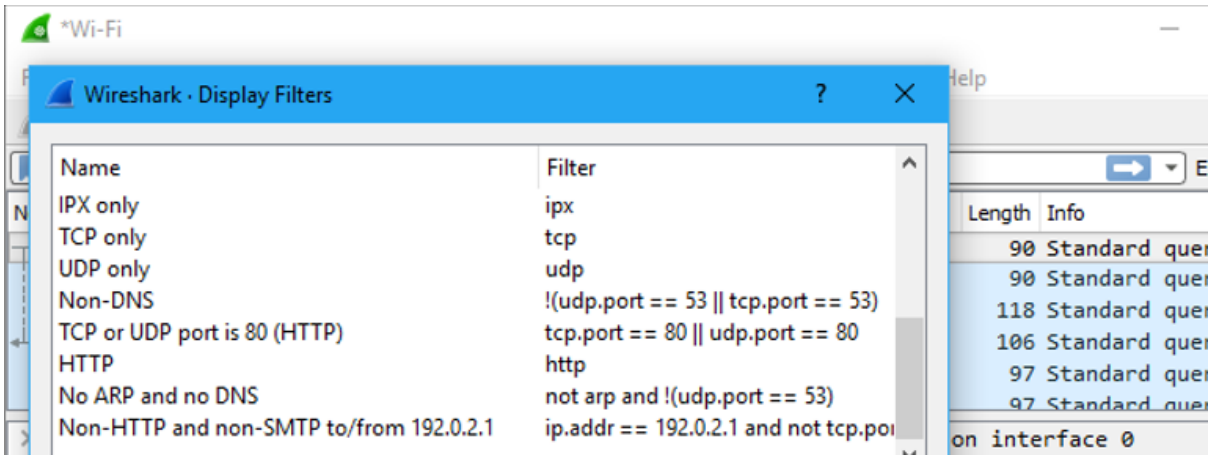


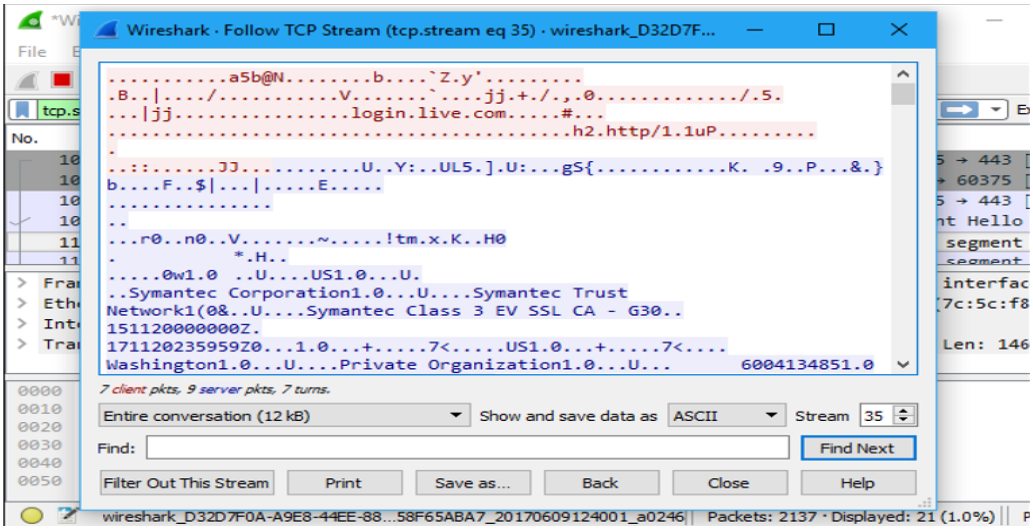


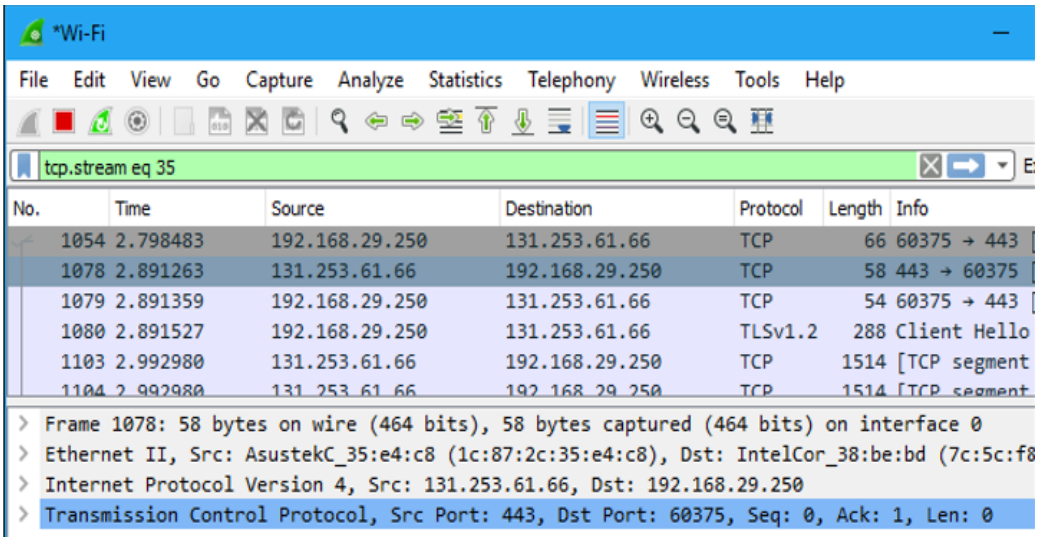

Capturing from Wi-Fi
File Edit
Wireshark · Coloring Rules · Default
Name
Filter
Bad TCP
tcp.analysis.flags && !tcp.analysis.window_upd
HSRP State Change
hsrp.state != 8 && hsrp.state != 16
Spanning Tree Topology Change
stp.type == 0x80
OSPF State Change
ospf.msg != 1
ICMP errors
icmp.type eq 3 || icmp.type eq 4 || icmp.type eq
ARP
arp
ICMP
icmp || icmpv6
TCP RST
tcp.flags.reset eq 1
SCTP ABORT
sctp.chunk_type eq ABORT
TTL low or unexpected
(! ip.dst == 224.0.0.0/4 && ip.ttl < 5 && !pim &
Checksum Errors
eth.fcs.status=="Bad" || ip.checksum.status=="
No.
Info
Router Advert
54656 → 27017
59500 → 443
443 → 59500
Frame
Ether
Inter
User
terface 0
:c8 (1c:87:2c

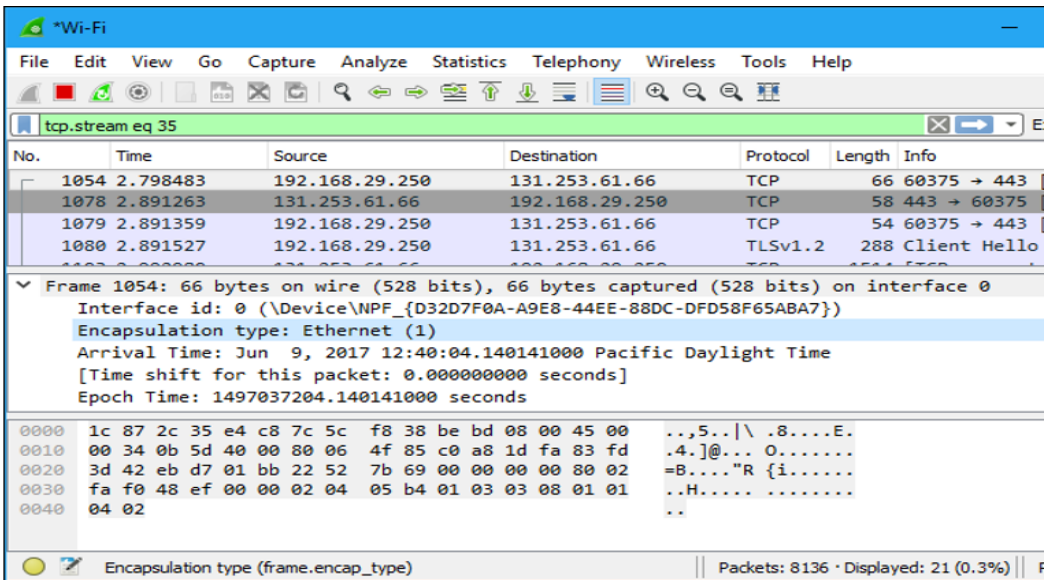

The Wireshark Network Analyzer
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Open Ctrl+O
Open Recent
Merge...
Import from Hex Dump...
Close Ctrl+W
Save Ctrl+S
Save As... Ctrl+Shift+S

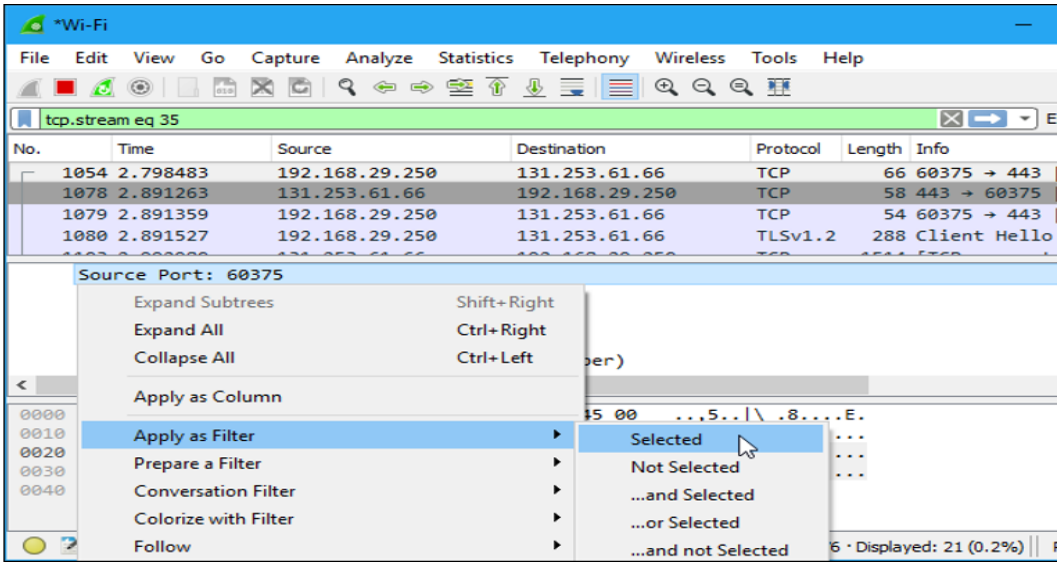

*Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
dns
No. Time Source Destination Protocol Length Info
305 5.248733 2601:1c0:cf00:8961:… 2601:1c0:cf00:8961:… DNS 90 Standard que
306 5.249092 2601:1c0:cf00:8961:… 2601:1c0:cf00:8961:… DNS 90 Standard que
307 5.269967 2601:1c0:cf00:8961:… 2601:1c0:cf00:8961:… DNS 118 Standard que
308 5.270325 2601:1c0:cf00:8961:… 2601:1c0:cf00:8961:… DNS 106 Standard que

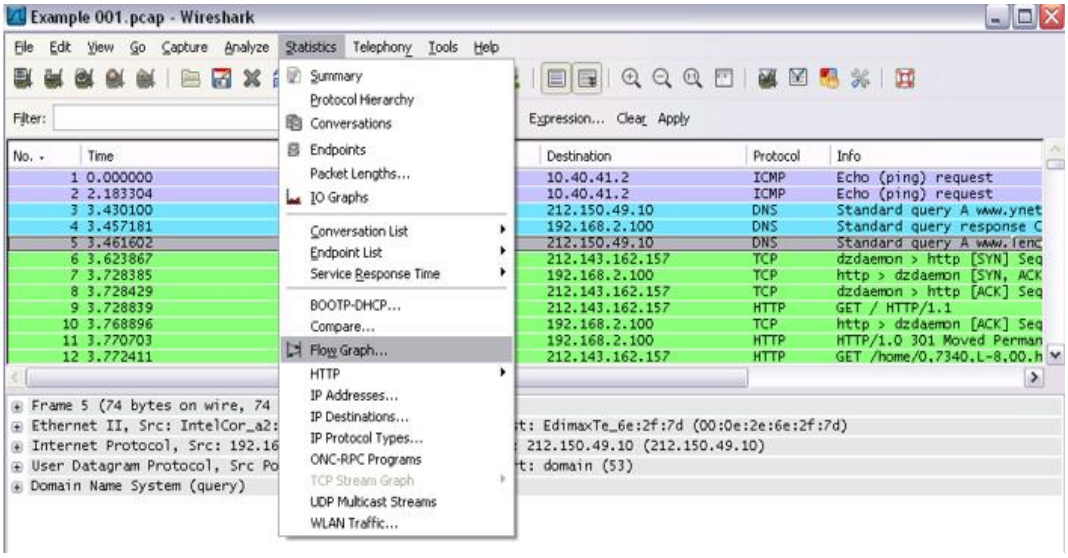

*Wi-Fi
Wireshark · Display Filters
Name
Filter
IPX only
ipx
TCP only
tcp
UDP only
udp
Non-DNS
!(udp.port == 53 || tcp.port == 53)
TCP or UDP port is 80 (HTTP)
tcp.port == 80 || udp.port == 80
HTTP
http
No ARP and no DNS
not arp and !(udp.port == 53)
Non-HTTP and non-SMTP to/from 192.0.2.1
ip.addr == 192.0.2.1 and not tcp.po
Length Info
90 Standard que
90 Standard que
118 Standard que
106 Standard que
97 Standard que
on interface 0

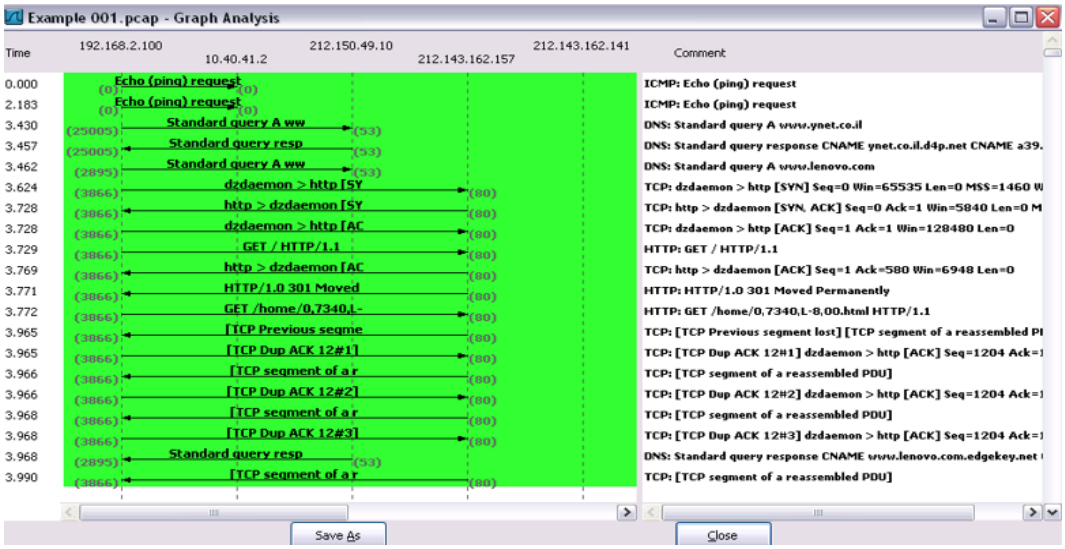

Wireshark · Follow TCP Stream (tcp.stream eq 35) · wireshark_D32D7F...
.a5b@N.....b....`Z.y'.....
.B..|..../.....V.....`....jj.+./.,.0.....
...|jj.....login.live.com.....#...
.....h2.http/1.1uP.....
..::.....JJ.....U..Y:..UL5.].U:...gS{.....K. .9..P...&.}
b....F..$|...|.....E.....
..
...r0..n0..V.....~.....!tm.x.K..H0
*.H..
.....0w1.0 ..U....US1.0...U.
..Symantec Corporation1.0...U....Symantec Trust
Network1(0&..U....Symantec Class 3 EV SSL CA - G30..
151120000000Z.
17112023595959Z0...1.0...+.....7<.....US1.0...+.....7<....
Washington1.0...U....Private Organization1.0...U... 6004134851.0
7 client pkts, 9 server pkts, 7 turns.
Entire conversation (12 kB)
Show and save data as
ASCII
Stream
35
Find:
Find Next
Filter Out This Stream
Print
Save as...
Back
Close
Help
wireshark_D32D7F0A-A9E8-44EE-88...58F65ABA7_20170609124001_a0246
Packets: 2137 · Displayed: 21 (1.0%)

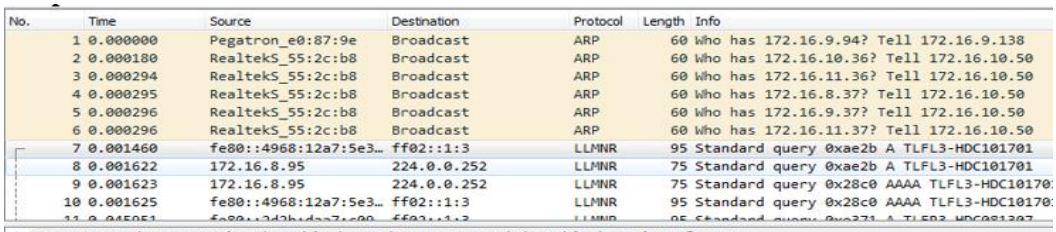

*Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.stream eq 35
No. Time Source Destination Protocol Length Info
1054 2.798483 192.168.29.250 131.253.61.66 TCP 66 60375 → 443
1078 2.891263 131.253.61.66 192.168.29.250 TCP 58 443 → 60375
1079 2.891359 192.168.29.250 131.253.61.66 TCP 54 60375 → 443
1080 2.891527 192.168.29.250 131.253.61.66 TLSv1.2 288 Client Hello
1103 2.992980 131.253.61.66 192.168.29.250 TCP 1514 [TCP segment
Frame 1078: 58 bytes on wire (464 bits), 58 bytes captured (464 bits) on interface 0
Ethernet II, Src: AsustekC_35:e4:c8 (1c:87:2c:35:e4:c8), Dst: IntelCor_38:be:bd (7c:5c:f8
Internet Protocol Version 4, Src: 131.253.61.66, Dst: 192.168.29.250
Transmission Control Protocol, Src Port: 443, Dst Port: 60375, Seq: 0, Ack: 1, Len: 0


*Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.stream eq 35
No. Time Source Destination Protocol Length Info
1054 2.798483 192.168.29.250 131.253.61.66 TCP 66 60375 → 443
1078 2.891263 131.253.61.66 192.168.29.250 TCP 58 443 → 60375
1079 2.891359 192.168.29.250 131.253.61.66 TCP 54 60375 → 443
1080 2.891527 192.168.29.250 131.253.61.66 TLSv1.2 288 Client Hello
Frame 1054: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0
Interface id: 0 (\Device\NPF_{D32D7F0A-A9E8-44EE-88DC-DFD58F65ABA7})
Encapsulation type: Ethernet (1)
Arrival Time: Jun 9, 2017 12:40:04.140141000 Pacific Daylight Time
[Time shift for this packet: 0.000000000 seconds]
Epoch Time: 1497037204.140141000 seconds
0000 1c 87 2c 35 e4 c8 7c 5c f8 38 be bd 08 00 45 00 ..,5..|\ .8....E.
0010 00 34 0b 5d 40 00 80 06 4f 85 c0 a8 1d fa 83 fd .4.]@... O.....
0020 3d 42 eb d7 01 bb 22 52 7b 69 00 00 00 00 80 02 =B....."R {i.....
0030 fa f0 48 ef 00 00 02 04 05 b4 01 03 03 08 01 01 ..H..... .....
0040 04 02 ..
Encapsulation type (frame.encap_type)
Packets: 8136 · Displayed: 21 (0.3%)


*Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.stream eq 35
No. Time Source Destination Protocol Length Info
1054 2.798483 192.168.29.250 131.253.61.66 TCP 66 60375 → 443
1078 2.891263 131.253.61.66 192.168.29.250 TCP 58 443 → 60375
1079 2.891359 192.168.29.250 131.253.61.66 TCP 54 60375 → 443
1080 2.891527 192.168.29.250 131.253.61.66 TLSv1.2 288 Client Hello
Source Port: 60375
Expand Subtrees Shift+Right
Expand All Ctrl+Right
Collapse All Ctrl+Left
Apply as Column
Apply as Filter
Prepare a Filter
Conversation Filter
Colorize with Filter
Follow
Selected
Not Selected
...and Selected
...or Selected
...and not Selected
Displayed: 21 (0.2%)


Example 001.pcap - Wireshark
File Edit View Go Capture Analyze Statistics Telephony Tools Help
Filter:
Expression... Clear Apply
No. Time Destination Protocol Info
1 0.000000 10.40.41.2 ICMP Echo (ping) request
2 2.183304 10.40.41.2 ICMP Echo (ping) request
3 3.430100 212.150.49.10 DNS Standard query A www.ynet
4 3.457181 192.168.2.100 DNS Standard query response C
5 3.461602 212.150.49.10 DNS Standard query A www.lenc
6 3.623867 212.143.162.157 TCP dzdaemon > http [SYN] Seq
7 3.728385 192.168.2.100 TCP http > dzdaemon [SYN, ACK
8 3.728429 212.143.162.157 TCP dzdaemon > http [ACK] Seq
9 3.728839 212.143.162.157 HTTP GET / HTTP/1.1
10 3.768896 192.168.2.100 TCP http > dzdaemon [ACK] Seq
11 3.770703 192.168.2.100 HTTP HTTP/1.0 301 Moved Perman
12 3.772411 212.143.162.157 HTTP GET /home/0,7340,L-8,00.h
Frame 5 (74 bytes on wire, 74
Ethernet II, Src: IntelCor_a2:
Internet Protocol, Src: 192.16
User Datagram Protocol, Src Po
Domain Name System (query)
Summary
Protocol Hierarchy
Conversations
Endpoints
Packet Lengths...
IO Graphs
Conversation List
Endpoint List
Service Response Time
BOOTP-DHCP...
Compare...
Flow Graph...
HTTP
IP Addresses...
IP Destinations...
IP Protocol Types...
ONC-RPC Programs
TCP Stream Graph
UDP Multicast Streams
WLAN Traffic...


Example 001.pcap - Graph Analysis
Time
192.168.2.100
10.40.41.2
212.150.49.10
212.143.162.157
212.143.162.141
Comment
0.000 Echo (ping) request
2.183 Echo (ping) request
3.430 Standard query A ww
3.457 Standard query resp
3.462 Standard query A ww
3.624 dzdaemon > http [SY
3.728 http > dzdaemon [SY
3.728 dzdaemon > http [AC
3.729 GET / HTTP/1.1
3.769 http > dzdaemon [AC
3.771 HTTP/1.0 301 Moved
3.772 GET /home/0,7340,L-
3.965 [TCP Previous segme
3.965 [TCP Dup ACK 12#1]
3.966 [TCP segment of a r
3.966 [TCP Dup ACK 12#2]
3.968 [TCP segment of a r
3.968 [TCP Dup ACK 12#3]
3.968 Standard query resp
3.990 [TCP segment of a r
ICMP: Echo (ping) request
ICMP: Echo (ping) request
DNS: Standard query A www.ynet.co.il
DNS: Standard query response CNAME ynet.co.il.d4p.net CNAME a39.
DNS: Standard query A www.lenovo.com
TCP: dzdaemon > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 W
TCP: http > dzdaemon [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 M
TCP: dzdaemon > http [ACK] Seq=1 Ack=1 Win=128480 Len=0
HTTP: GET / HTTP/1.1
TCP: http > dzdaemon [ACK] Seq=1 Ack=580 Win=6948 Len=0
HTTP: HTTP/1.0 301 Moved Permanently
HTTP: GET /home/0,7340,L-8,00.html HTTP/1.1
TCP: [TCP Previous segment lost] [TCP segment of a reassembled PI
TCP: [TCP Dup ACK 12#1] dzdaemon > http [ACK] Seq=1204 Ack=
TCP: [TCP segment of a reassembled PDU]
TCP: [TCP Dup ACK 12#2] dzdaemon > http [ACK] Seq=1204 Ack=
TCP: [TCP segment of a reassembled PDU]
TCP: [TCP Dup ACK 12#3] dzdaemon > http [ACK] Seq=1204 Ack=
DNS: Standard query response CNAME www.lenovo.com.edgekey.net
TCP: [TCP segment of a reassembled PDU]
Save As
Close



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	Pegatron_e0:87:9e	Broadcast	ARP	60	Who has 172.16.9.94? Tell 172.16.9.138
2	0.000180	RealtekS_55:2c:b8	Broadcast	ARP	60	Who has 172.16.10.36? Tell 172.16.10.50
3	0.000294	RealtekS_55:2c:b8	Broadcast	ARP	60	Who has 172.16.11.36? Tell 172.16.10.50
4	0.000295	RealtekS_55:2c:b8	Broadcast	ARP	60	Who has 172.16.8.37? Tell 172.16.10.50
5	0.000296	RealtekS_55:2c:b8	Broadcast	ARP	60	Who has 172.16.9.37? Tell 172.16.10.50
6	0.000296	RealtekS_55:2c:b8	Broadcast	ARP	60	Who has 172.16.11.37? Tell 172.16.10.50
7	0.001460	fe80::4968:12a7:5e3…	ff02::1:3	LLMNR	95	Standard query 0xae2b A TLFL3-HDC101701
8	0.001622	172.16.8.95	224.0.0.252	LLMNR	75	Standard query 0xae2b A TLFL3-HDC101701
9	0.001623	172.16.8.95	224.0.0.252	LLMNR	75	Standard query 0x28c0 AAAA TLFL3-HDC10170
10	0.001625	fe80::4968:12a7:5e3…	ff02::1:3	LLMNR	95	Standard query 0x28c0 AAAA TLFL3-HDC10170

```
▷ Frame 7: 95 bytes on wire (760 bits), 95 bytes captured (760 bits) on interface 0
▷ Ethernet II, Src: Dell_35:10:a8 (50:9a:4c:35:10:a8), Dst: IPv6mcast_01:00:03 (33:33:00:01:00:03)
▷ Internet Protocol Version 6, Src: fe80::4968:12a7:5e36:523e, Dst: ff02::1:3
◢ User Datagram Protocol, Src Port: 62374, Dst Port: 5355
    Source Port: 62374
    Destination Port: 5355
    Length: 41
    Checksum: 0x90e0 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 0]
▷ Link-local Multicast Name Resolution (query)
```

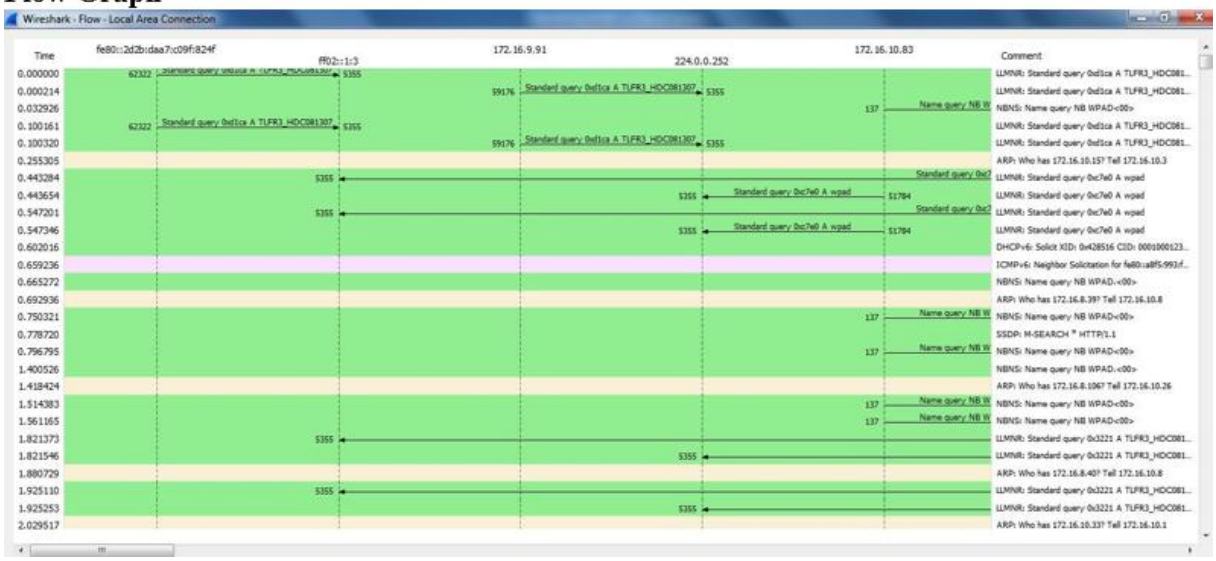
```
0000  33 33 00 01 00 03 50 9a  4c 35 10 a8 86 dd 60 00   33····P· L5····`·
0010  00 00 00 29 11 01 fe 80  00 00 00 00 00 00 49 68   ···)···· ·····Ih
0020  12 a7 5e 36 52 3e ff 02  00 00 00 00 00 00 00 00   ··^6R>·· ········
0030  00 00 00 01 00 03 f3 a6  14 eb 00 29 90 e0 ae 2b   ········ ···)···+
0040  00 00 00 01 00 00 00 00  00 00 0f 54 4c 46 4c 33   ········ ···TLFL3
0050  2d 48 44 43 31 30 31 37  30 31 00 00 01 00 01      -HDC1017 01·····
```

No.	Time	Source	Destination	Protocol	Length	Info
123	4.557832	fe80::8532:3a9f:aff…	fe80::5c2b:19eb:d33…	TCP	74	1509 → 2869 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
126	4.557993	172.16.9.106	172.16.9.96	TCP	60	1506 → 2869 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1095	30.718732	172.16.8.83	172.16.9.96	TCP	66	51526 → 2869 [SYN, ECN, CWR] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
1096	30.718794	172.16.9.96	172.16.8.83	TCP	66	2869 → 51526 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
1097	30.719129	172.16.8.83	172.16.9.96	TCP	60	51526 → 2869 [ACK] Seq=1 Ack=1 Win=65536 Len=0
1099	30.719919	172.16.9.96	172.16.8.83	TCP	278	2869 → 51526 [PSH, ACK] Seq=1 Ack=133 Win=65536 Len=224 [TCP segment of a reassembled PDU]
1100	30.719986	172.16.9.96	172.16.8.83	TCP	1514	2869 → 51526 [ACK] Seq=225 Ack=133 Win=65536 Len=1460 [TCP segment of a reassembled PDU]
1101	30.720279	172.16.8.83	172.16.9.96	TCP	60	51526 → 2869 [ACK] Seq=133 Ack=1685 Win=65536 Len=0

```
▷ Frame 123: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0
▷ Ethernet II, Src: RealtekS_b2:60:90 (00:e0:4c:b2:60:90), Dst: IntelCor_13:ed:7c (00:27:0e:13:ed:7c)
▷ Internet Protocol Version 6, Src: fe80::8532:3a9f:aff1:b3ca, Dst: fe80::5c2b:19eb:d33d:a1cd
▷ Transmission Control Protocol, Src Port: 1509, Dst Port: 2869, Seq: 1, Ack: 1, Len: 0
```

```
0000  00 27 0e 13 ed 7c 00 e0  4c b2 60 90 86 dd 60 00   ·'···|·· L·`···`·
0010  00 00 00 14 06 80 fe 80  00 00 00 00 00 00 85 32   ···· ···· ·····2
0020  3a 9f af f1 b3 ca fe 80  00 00 00 00 00 00 5c 2b   :····· ·····\+
0030  19 eb d3 3d a1 cd 05 e5  0b 35 3b ef f1 2f bf d2   ···=···· ·5;··/··
0040  67 35 50 14 00 00 3e de  00 00                    g5P···>· ··
```

Flow Graph

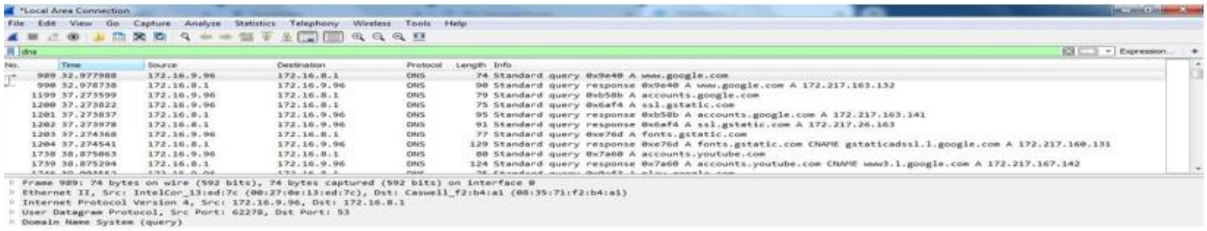


arp

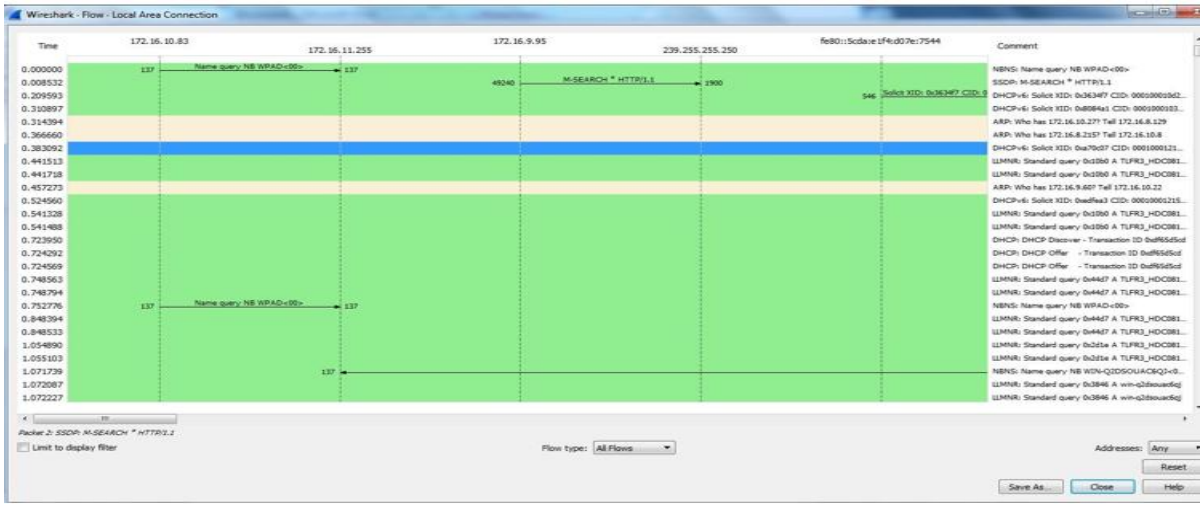
No.	Time	Source	Destination	Protocol	Length	Info
6	0.255305	Foxconn_c9:c5:f0	Broadcast	ARP	60	Who has 172.16.10.15? Tell 172.16.10.3
14	0.692936	Foxconn_d0:ac:46	Broadcast	ARP	60	Who has 172.16.8.39? Tell 172.16.10.8
19	1.418424	Foxconn_c9:c9:91	Broadcast	ARP	60	Who has 172.16.8.106? Tell 172.16.10.26
24	1.880729	Foxconn_d0:ac:46	Broadcast	ARP	60	Who has 172.16.8.40? Tell 172.16.10.8
27	2.029517	Giga-Byt_92:d2:ef	Broadcast	ARP	60	Who has 172.16.10.33? Tell 172.16.10.1
41	2.509905	Giga-Byt_7c:c5:34	Broadcast	ARP	60	Who has 172.16.9.82? Tell 172.16.9.111
44	2.602358	Foxconn_c9:c8:24	Broadcast	ARP	60	Who has 172.16.8.139? Tell 172.16.10.22
46	2.743021	Dell_35:11:11	Broadcast	ARP	60	Who has 172.16.8.118? Tell 172.16.10.195
56	3.201822	Giga-Byt_92:d2:ef	Broadcast	ARP	60	Who has 172.16.10.34? Tell 172.16.10.1
60	3.237061	Giga-Byt_7c:c5:34	Broadcast	ARP	60	Who has 172.16.9.82? Tell 172.16.9.111

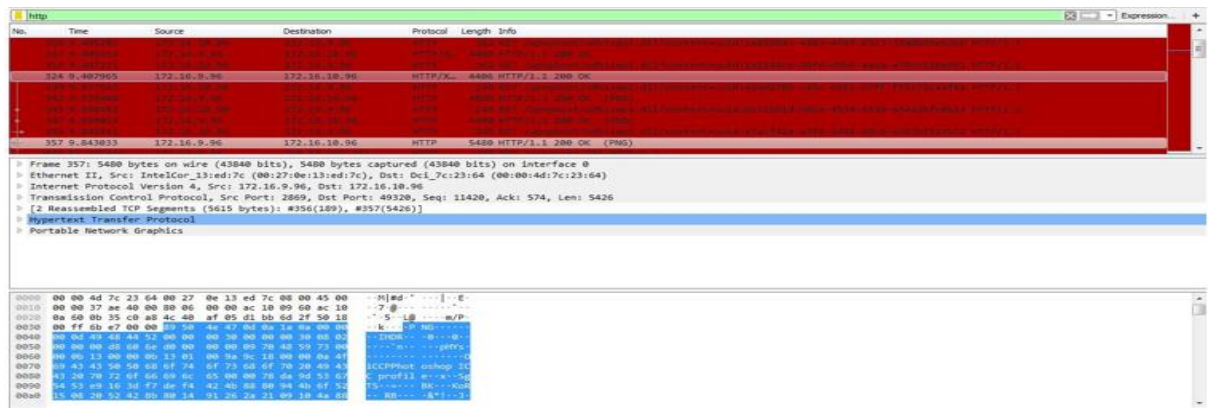
```
Frame 119: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface 0
Ethernet II, Src: IntelCor_13:ed:7c (00:27:0e:13:ed:7c), Dst: RealtekS_b2:60:90 (00:e0:4c:b2:60:90)
Address Resolution Protocol (reply)
```

```
0000  00 e0 4c b2 60 90 00 27  0e 13 ed 7c 08 06 00 01   ··L·`··'  ···|····
0010  08 00 06 04 00 02 00 27  0e 13 ed 7c ac 10 09 60   ·······'  ···|···`
0020  00 e0 4c b2 60 90 ac 10  09 6a                     ··L·`···  ·j
```

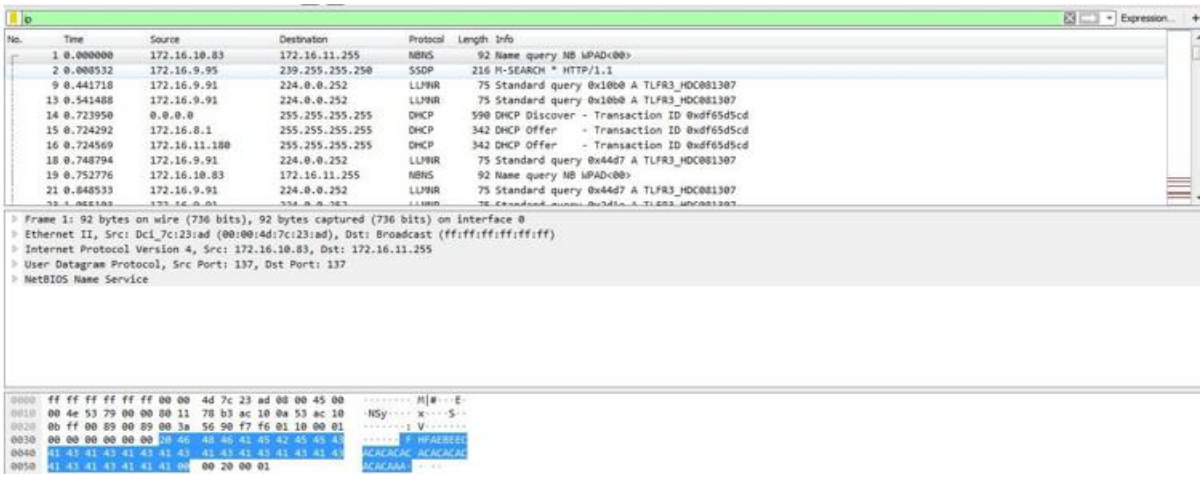


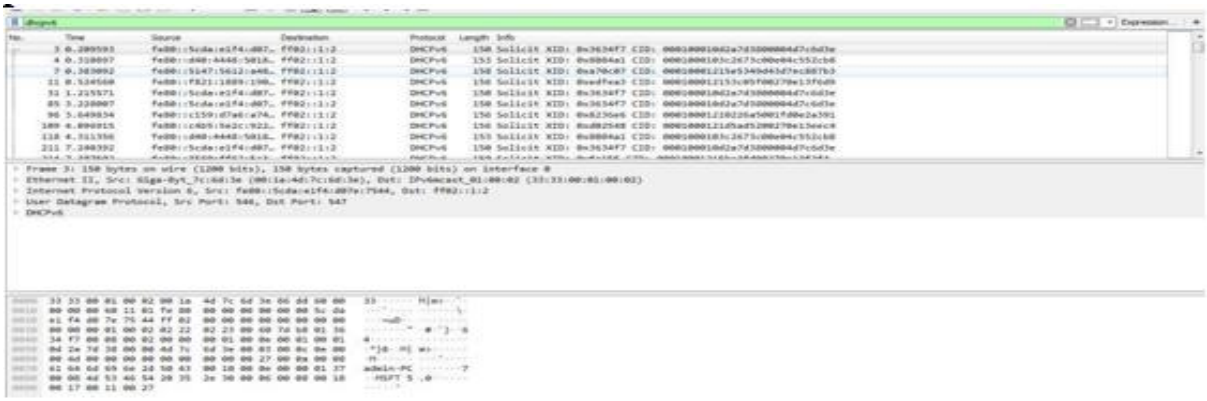

```
0000  08 35 71 f2 b4 a1 00 27  0e 13 ed 7c 08 00 45 00   ·5q····'  ···|··E·
0010  00 3c 37 bb 00 00 80 11  00 00 ac 10 09 60 ac 10   ·<7·····  ·····`··
0020  08 01 f3 46 00 35 00 28  69 bb 9e 40 01 00 00 01   ···F·5·(  i··@····
0030  00 00 00 00 00 00 03 77  77 77 06 67 6f 6f 67 6c   ·······w  ww·googl
0040  65 03 63 6f 6d 00 00 01  00 01                     e·com···  ··
```

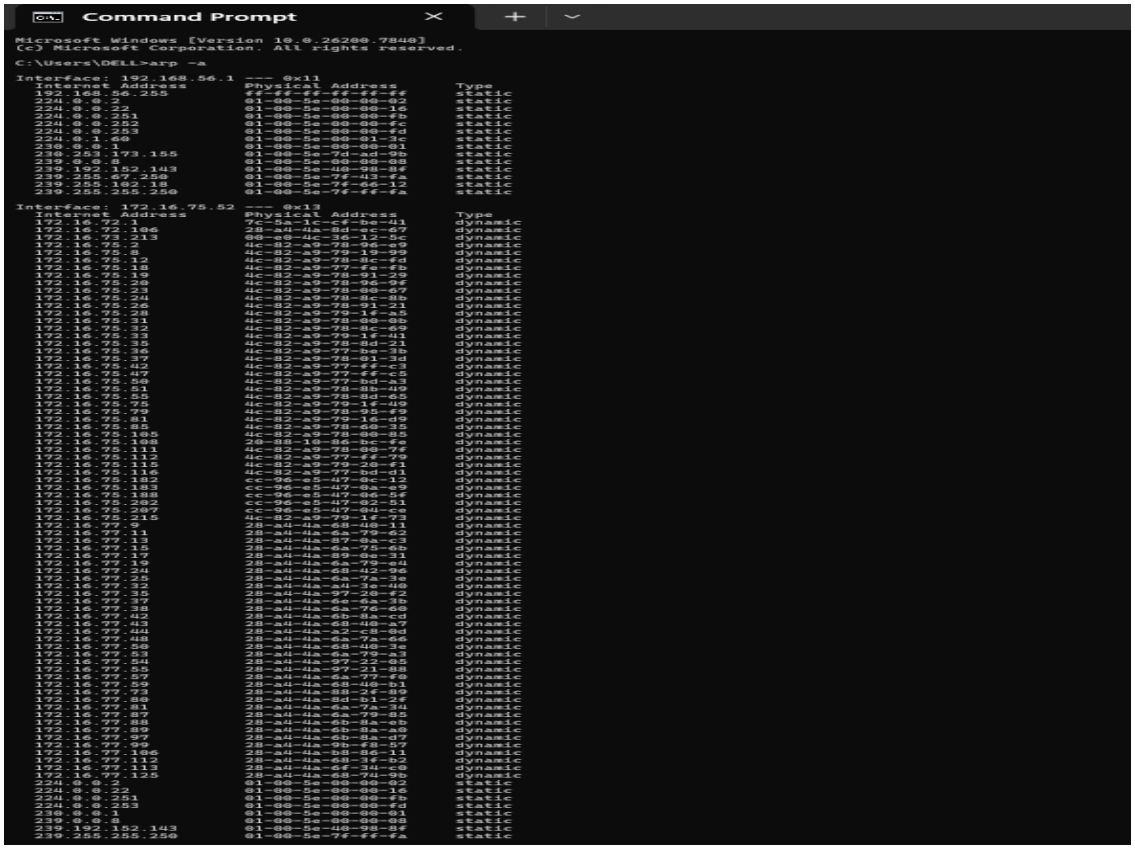


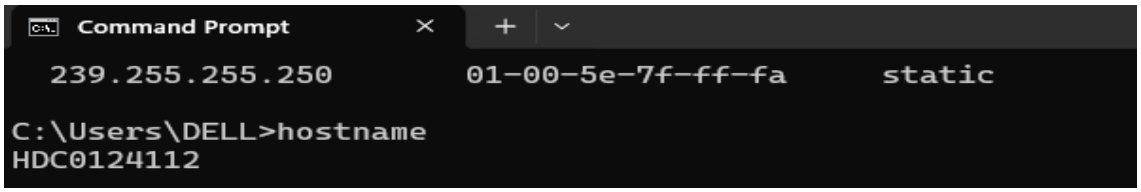



```
0000  00 00 4d 7c 23 64 00 27  0e 13 ed 7c 08 00 45 00   ··M|#d·'  ···|··E·
0010  00 00 37 ae 40 00 80 06  00 00 ac 10 09 60 ac 10   ··7·@···  ·····`··
0020  0a 60 0b 35 c0 a8 4c 48  af 05 d1 bb 6d 2f 50 18   ·`·5··LH  ····m/P·
0030  00 ff 6b e7 00 00 89 50  4e 47 0d 0a 1a 0a 00 00   ··k····P  NG······
0040  00 0d 49 48 44 52 00 00  00 30 00 00 00 30 08 02   ··IHDR··  ·0···0··
0050  00 00 00 d8 68 6e 00 00  00 09 70 48 59 73 00 00   ····hn··  ··pHYs··
0060  0e c3 00 00 0e c3 01 c7  6f a8 64 00 00 0a 4f 69   ········  o·d···Oi
0070  43 43 50 50 68 6f 74 6f  73 68 6f 70 20 49 43 43   CCPPhoto  shop ICC
0080  20 70 72 6f 66 69 6c 65  00 00 78 da 9d 53 67 54   profile  ··x··SgT
0090  53 e9 16 3d f7 de f4 42  4b 88 80 94 4b 6f 52 15   S··=···B  K···KoR·
00a0  40 81 20 52 42 8b 80 14  91 26 2a 21 09 10 4a 88   @· RB····  ·&*!··J·
```


No. Time Source Destination Protocol Length Info
1 0.000000 172.16.10.83 172.16.11.255 NBNS 92 Name query NB WPAD<00>
2 0.000532 172.16.9.95 239.255.255.250 SSDP 216 M-SEARCH * HTTP/1.1
9 0.441718 172.16.9.91 224.0.0.252 LLMNR 75 Standard query 0x10b0 A TLFR3_HDC081307
13 0.541488 172.16.9.91 224.0.0.252 LLMNR 75 Standard query 0x10b0 A TLFR3_HDC081307
14 0.723950 0.0.0.0 255.255.255.255 DHCP 590 DHCP Discover - Transaction ID 0xdf65d5cd
15 0.724292 172.16.8.1 255.255.255.255 DHCP 342 DHCP Offer - Transaction ID 0xdf65d5cd
16 0.724569 172.16.11.180 255.255.255.255 DHCP 342 DHCP Offer - Transaction ID 0xdf65d5cd
18 0.748794 172.16.9.91 224.0.0.252 LLMNR 75 Standard query 0x44d7 A TLFR3_HDC081307
19 0.752776 172.16.10.83 172.16.11.255 NBNS 92 Name query NB WPAD<00>
21 0.848533 172.16.9.91 224.0.0.252 LLMNR 75 Standard query 0x44d7 A TLFR3_HDC081307
Frame 1: 92 bytes on wire (736 bits), 92 bytes captured (736 bits) on interface 0
Ethernet II, Src: Dci_7c:23:ad (00:00:4d:7c:23:ad), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
Internet Protocol Version 4, Src: 172.16.10.83, Dst: 172.16.11.255
User Datagram Protocol, Src Port: 137, Dst Port: 137
NetBIOS Name Service
0000 ff ff ff ff ff ff 00 00 4d 7c 23 ad 08 00 45 00
0010 00 4e 53 79 00 00 80 11 78 b3 ac 10 0a 53 ac 10
0020 0b ff 00 89 00 89 00 3a 56 90 f7 f6 01 10 00 01
0030 00 00 00 00 00 00 20 46 48 46 41 45 42 45 45 42
0040 41 43 41 43 41 43 41 43 41 43 41 43 41 43 41 43
0050 41 43 41 43 41 41 41 00 00 20 00 01


No. Time Source Destination Protocol Length Info
3 0.280593 fe80::5cda:e1f4:d87… ff02::1:2 DHCPv6 150 Solicit XID: 0x3634f7 CID: 000100010d2e7d53000004d7c6d3e
4 0.310897 fe80::d40:4448:5818… ff02::1:2 DHCPv6 155 Solicit XID: 0x8804a1 CID: 000100010d3c2675c00e04c552cb6
7 0.363092 fe80::5147:5612:a48… ff02::1:2 DHCPv6 150 Solicit XID: 0xa70c87 CID: 000100012119e5340d43d7ec887b3
11 0.534508 fe80::f821:1889:198… ff02::1:2 DHCPv6 150 Solicit XID: 0xedfea3 CID: 000100012153c05f00279e13fbd8
31 1.215571 fe80::5cda:e1f4:d87… ff02::1:2 DHCPv6 150 Solicit XID: 0x3634f7 CID: 000100010d2e7d53000004d7c6d3e
85 3.220807 fe80::5cda:e1f4:d87… ff02::1:2 DHCPv6 150 Solicit XID: 0x3634f7 CID: 000100010d2e7d53000004d7c6d3e
96 5.049834 fe80::c159:d7e6:e74… ff02::1:2 DHCPv6 156 Solicit XID: 0x8236e6 CID: 000100012102626e5001fd0e2e391
189 6.890815 fe80::c405:9e2c:922… ff02::1:2 DHCPv6 156 Solicit XID: 0xd825e8 CID: 000100012105ad52002279e13eec8
118 4.311356 fe80::d40:4448:5818… ff02::1:2 DHCPv6 155 Solicit XID: 0x8804a1 CID: 000100010d3c2675c00e04c552cb6
211 7.248392 fe80::5cda:e1f4:d87… ff02::1:2 DHCPv6 150 Solicit XID: 0x3634f7 CID: 000100010d2e7d53000004d7c6d3e
Frame 3: 150 bytes on wire (1200 bits), 150 bytes captured (1200 bits) on interface 0
Ethernet II, Src: Giga-Byt_7c:6d:3e (00:1a:4d:7c:6d:3e), Dst: IPv6mcast_01:00:02 (33:33:00:01:00:02)
Internet Protocol Version 6, Src: fe80::5cda:e1f4:d87e:7544, Dst: ff02::1:2
User Datagram Protocol, Src Port: 546, Dst Port: 547
DHCPv6






```
C:\Users\DELL>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::2709:30a3:dc42:768d%17
   IPv4 Address. . . . . . . . . . . : 192.168.56.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Wireless LAN adapter Local Area Connection* 9:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 10:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter WiFi:

   Connection-specific DNS Suffix  . :
   IPv4 Address. . . . . . . . . . . : 172.16.75.52
   Subnet Mask . . . . . . . . . . . : 255.255.248.0
   Default Gateway . . . . . . . . . : 172.16.72.1
```

```
C:\Users\DELL>ipconfig /all

Windows IP Configuration

   Host Name . . . . . . . . . . . . : HDC0124112
   Primary Dns Suffix  . . . . . . . :
   Node Type . . . . . . . . . . . . : Hybrid
   IP Routing Enabled. . . . . . . . : No
   WINS Proxy Enabled. . . . . . . . : No

Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Intel(R) Ethernet Connection (17) I219-LM
   Physical Address. . . . . . . . . : 20-88-10-86-C7-DB
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes

Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : VirtualBox Host-Only Ethernet Adapter
   Physical Address. . . . . . . . . : 0A-00-27-00-00-11
   DHCP Enabled. . . . . . . . . . . : No
   Autoconfiguration Enabled . . . . : Yes
   Link-local IPv6 Address . . . . . : fe80::2709:30a3:dc42:768d%17(Preferred)
   IPv4 Address. . . . . . . . . . . : 192.168.56.1(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :
   DHCPv6 IAID . . . . . . . . . . . : 873070631
   DHCPv6 Client DUID. . . . . . . . : 00-01-01-00-31-4A-B6-03-20-88-10-86-C7-DB
   NetBIOS over Tcpip. . . . . . . . : Enabled

Wireless LAN adapter Local Area Connection* 9:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Microsoft Wi-Fi Direct Virtual Adapter
   Physical Address. . . . . . . . . : 4E-82-A9-78-90-37
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes

Wireless LAN adapter Local Area Connection* 10:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Microsoft Wi-Fi Direct Virtual Adapter #2
   Physical Address. . . . . . . . . : 42-82-A9-78-90-37
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes

Wireless LAN adapter WiFi:

   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Realtek RTL8852BE WiFi 6 802.11ax PCIe Adapter
   Physical Address. . . . . . . . . : 4C-82-A9-78-90-37
   DHCP Enabled. . . . . . . . . . . : No
   Autoconfiguration Enabled . . . . : Yes
   IPv4 Address. . . . . . . . . . . : 172.16.75.52(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.248.0
   Default Gateway . . . . . . . . . : 172.16.72.1
   DNS Servers . . . . . . . . . . . : 172.16.72.1
                                       8.8.8.8
   NetBIOS over Tcpip. . . . . . . . : Enabled
```

```
C:\Users\DELL>nbtstat -a

Displays protocol statistics and current TCP/IP connections using NBT
(NetBIOS over TCP/IP).

NBTSTAT [ [-a RemoteName] [-A IP address] [-c] [-n]
        [-r] [-R] [-RR] [-s] [-S] [interval] ]

  -a   (adapter status) Lists the remote machine's name table given its name
  -A   (Adapter status) Lists the remote machine's name table given its
                        IP address.
  -c   (cache)          Lists NBT's cache of remote [machine] names and their IP addresses
  -n   (names)          Lists local NetBIOS names.
  -r   (resolved)       Lists names resolved by broadcast and via WINS
  -R   (Reload)         Purges and reloads the remote cache name table
  -S   (Sessions)       Lists sessions table with the destination IP addresses
  -s   (sessions)       Lists sessions table converting destination IP
                        addresses to computer NETBIOS names.
  -RR  (ReleaseRefresh) Sends Name Release packets to WINS and then, starts Refresh

  RemoteName   Remote host machine name.
  IP address   Dotted decimal representation of the IP address.
  interval     Redisplays selected statistics, pausing interval seconds
               between each display. Press Ctrl+C to stop redisplaying
               statistics.
```

```
C:\Users\DELL>pathping

Usage: pathping [-g host-list] [-h maximum_hops] [-i address] [-n]
                [-p period] [-q num_queries] [-w timeout]
                [-4] [-6] target_name

Options:
    -g host-list     Loose source route along host-list.
    -h maximum_hops  Maximum number of hops to search for target.
    -i address       Use the specified source address.
    -n               Do not resolve addresses to hostnames.
    -p period        Wait period milliseconds between pings.
    -q num_queries   Number of queries per hop.
    -w timeout       Wait timeout milliseconds for each reply.
    -4               Force using IPv4.
    -6               Force using IPv6.
```

```
C:\Users\DELL>ping

Usage: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
            [-r count] [-s count] [[-j host-list] | [-k host-list]]
            [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
            [-4] [-6] target_name

Options:
    -t             Ping the specified host until stopped.
                   To see statistics and continue - type Control-Break;
                   To stop - type Control-C.
    -a             Resolve addresses to hostnames.
    -n count       Number of echo requests to send.
    -l size        Send buffer size.
    -f             Set Don't Fragment flag in packet (IPv4-only).
    -i TTL         Time To Live.
    -v TOS         Type Of Service (IPv4-only. This setting has been deprecated
                   and has no effect on the type of service field in the IP
                   Header).
    -r count       Record route for count hops (IPv4-only).
    -s count       Timestamp for count hops (IPv4-only).
    -j host-list   Loose source route along host-list (IPv4-only).
    -k host-list   Strict source route along host-list (IPv4-only).
    -w timeout     Timeout in milliseconds to wait for each reply.
    -R             Use routing header to test reverse route also (IPv6-only).
                   Per RFC 5095 the use of this routing header has been
                   deprecated. Some systems may drop echo requests if
                   this header is used.
    -S srcaddr     Source address to use.
    -c compartment Routing compartment identifier.
    -p             Ping a Hyper-V Network Virtualization provider address.
    -4             Force using IPv4.
    -6             Force using IPv6.
```

```
C:\Users\DELL>route

Manipulates network routing tables.

ROUTE [-f] [-p] [-4|-6] command [destination]
                  [MASK netmask]  [gateway] [METRIC metric]  [IF interface]

  -f           Clears the routing tables of all gateway entries.  If this is
               used in conjunction with one of the commands, the tables are
               cleared prior to running the command.

  -p           When used with the ADD command, makes a route persistent across
               boots of the system. By default, routes are not preserved
               when the system is restarted. Ignored for all other commands,
               which always affect the appropriate persistent routes.

  -4           Force using IPv4.

  -6           Force using IPv6.

  command      One of these:
                 PRINT     Prints  a route
                 ADD       Adds    a route
                 DELETE    Deletes a route
                 CHANGE    Modifies an existing route
  destination  Specifies the host.
  MASK         Specifies that the next parameter is the 'netmask' value.
  netmask      Specifies a subnet mask value for this route entry.
               If not specified, it defaults to 255.255.255.255.
  gateway      Specifies gateway.
  interface    the interface number for the specified route.
  METRIC       specifies the metric, ie. cost for the destination.

All symbolic names used for destination are looked up in the network database
file NETWORKS. The symbolic names for gateway are looked up in the host name
database file HOSTS.

If the command is PRINT or DELETE. Destination or gateway can be a wildcard,
(wildcard is specified as a star '*'), or the gateway argument may be omitted.

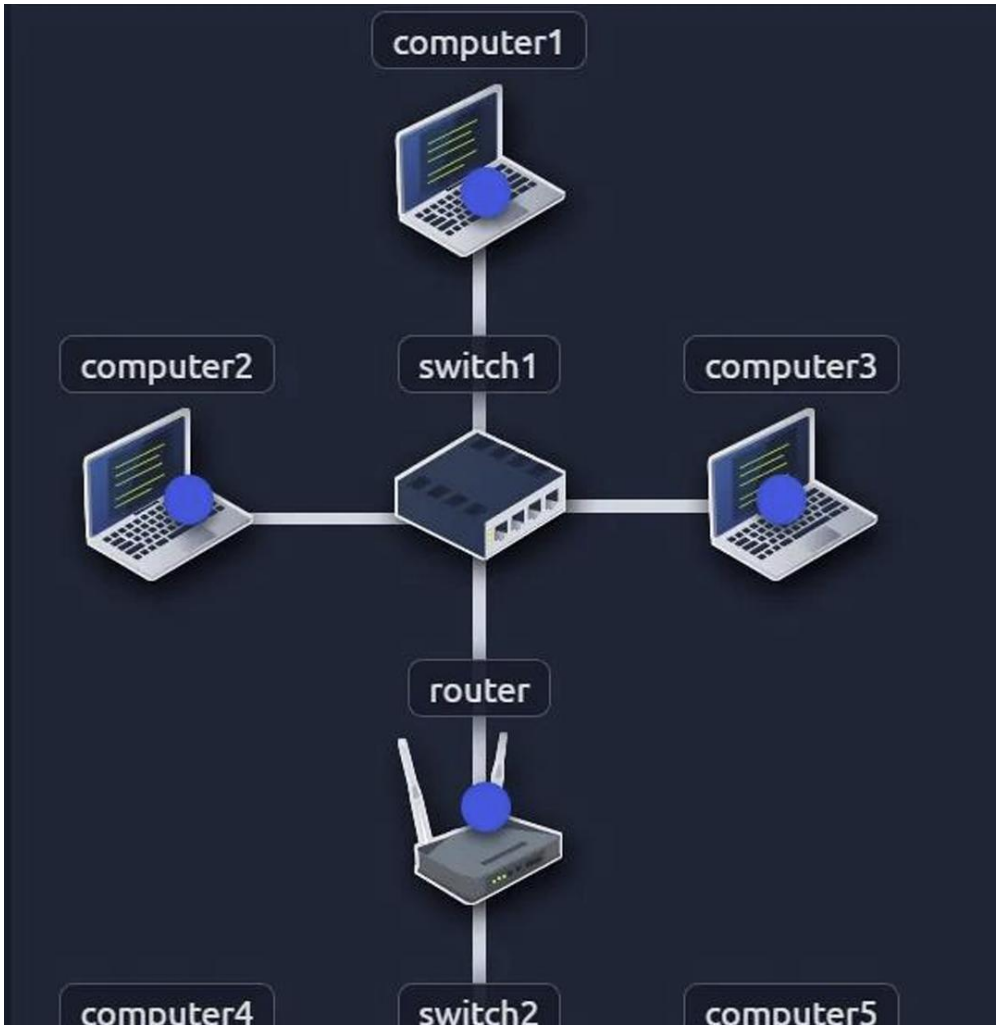
If Dest contains a * or ?, it is treated as a shell pattern, and only
matching destination routes are printed. The '*' matches any string,
and '?' matches any one char. Examples: 157.*.1, 157.*, 127.*, *224*.

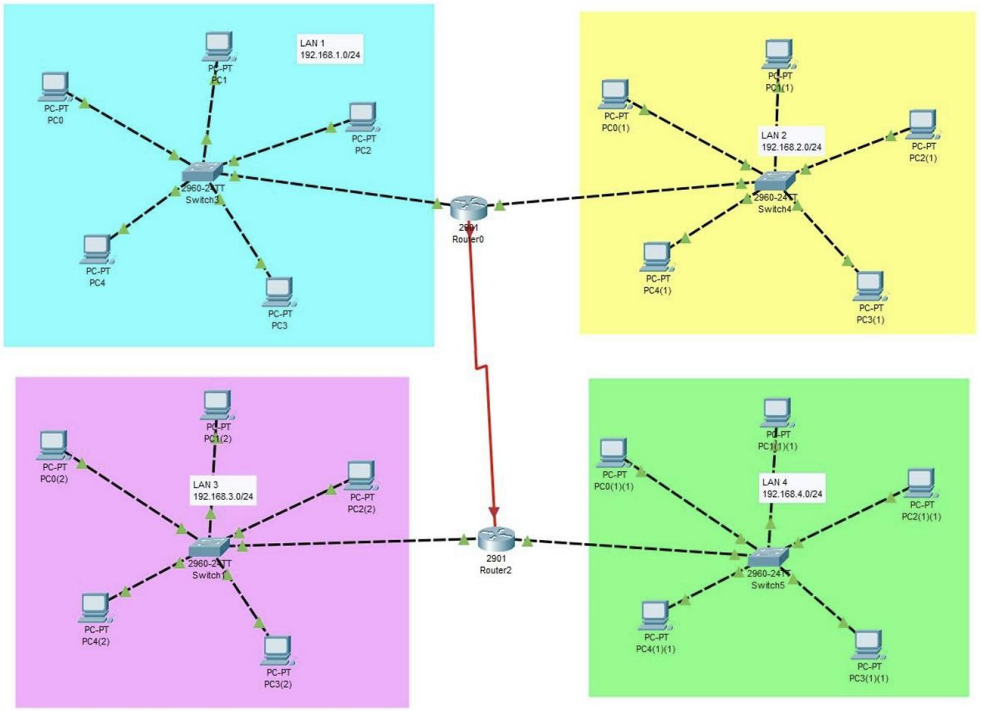
Pattern match is only allowed in PRINT command.
Diagnostic Notes:
    Invalid MASK generates an error, that is when (DEST & MASK) != DEST.
    Example> route ADD 157.0.0.0 MASK 155.0.0.0 157.55.80.1 IF 1
             The route addition failed: The specified mask parameter is invalid. (Destination & Mask) != Destination.
```

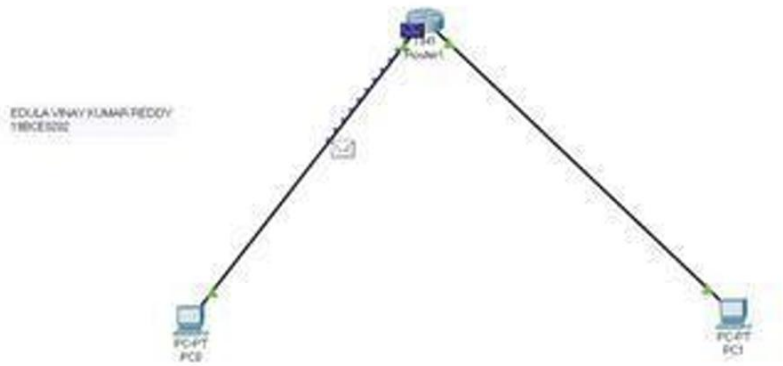
```
C:\Users\DELL>netstat

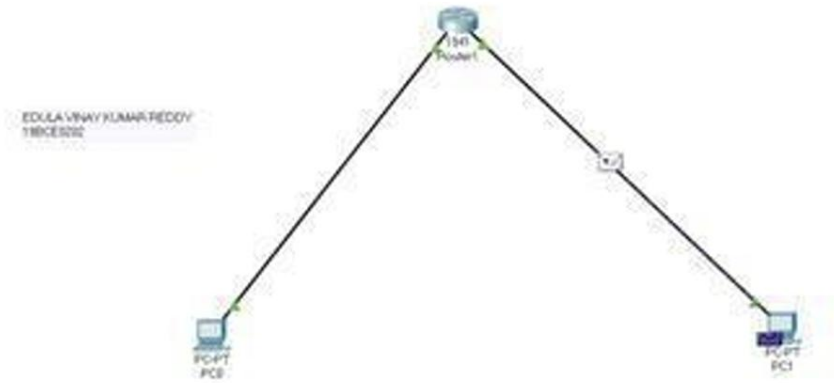
Active Connections

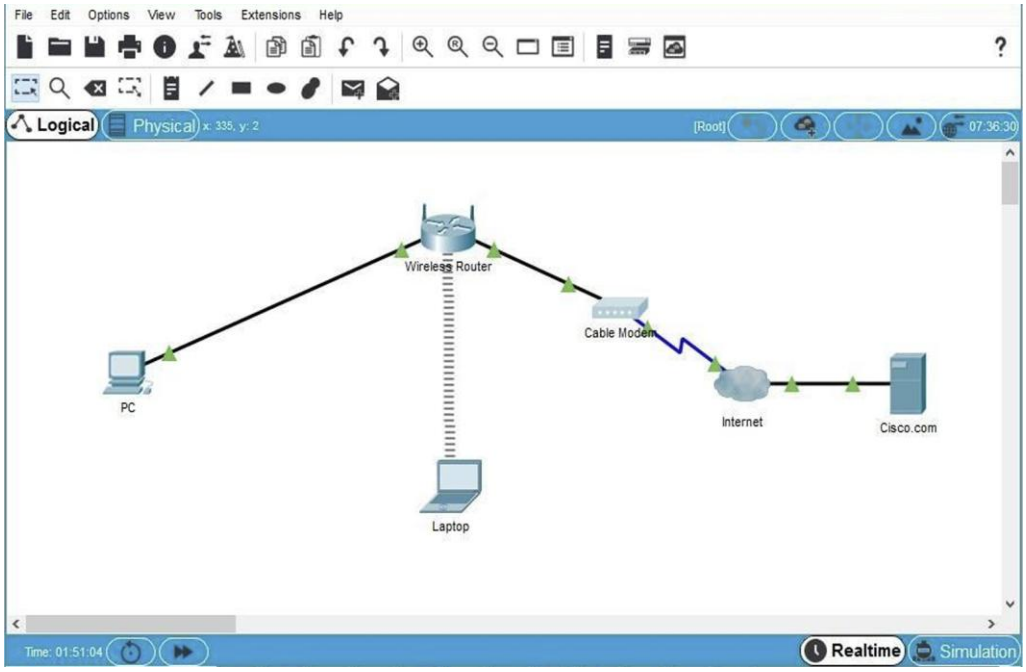
  Proto  Local Address          Foreign Address        State
  TCP    127.0.0.1:49675        HDC0124113:49676       ESTABLISHED
  TCP    127.0.0.1:49676        HDC0124113:49675       ESTABLISHED
  TCP    127.0.0.1:49677        HDC0124113:49678       ESTABLISHED
  TCP    127.0.0.1:49678        HDC0124113:49677       ESTABLISHED
  TCP    127.0.0.1:49682        HDC0124113:49683       ESTABLISHED
  TCP    127.0.0.1:49683        HDC0124113:49682       ESTABLISHED
  TCP    127.0.0.1:63945        HDC0124113:63944       TIME_WAIT
  TCP    172.16.75.53:1521      HDC0124113:57609       ESTABLISHED
  TCP    172.16.75.53:7680      172.16.75.35:49193     TIME_WAIT
  TCP    172.16.75.53:7680      172.16.75.85:60908     TIME_WAIT
  TCP    172.16.75.53:7680      172.16.75.105:58668    ESTABLISHED
  TCP    172.16.75.53:7680      172.16.77.24:55750     TIME_WAIT
  TCP    172.16.75.53:7680      172.16.77.33:56991     TIME_WAIT
  TCP    172.16.75.53:7680      172.16.77.34:49585     TIME_WAIT
  TCP    172.16.75.53:7680      172.16.77.37:59924     TIME_WAIT
  TCP    172.16.75.53:7680      THALAPATHY-VIJA:61829  TIME_WAIT
  TCP    172.16.75.53:49414     4.213.25.242:https     ESTABLISHED
  TCP    172.16.75.53:49964     pnmaaa-bf-in-f10:https  ESTABLISHED
  TCP    172.16.75.53:50044     sa-in-f188:5228        ESTABLISHED
  TCP    172.16.75.53:56658     172.16.75.115:ms-do    ESTABLISHED
  TCP    172.16.75.53:57609     HDC0124113:1521        ESTABLISHED
  TCP    172.16.75.53:59971     172.16.75.115:ms-do    ESTABLISHED
  TCP    172.16.75.53:60774     4.150.223.97:https     ESTABLISHED
  TCP    172.16.75.53:60779     172.16.75.115:ms-do    ESTABLISHED
  TCP    172.16.75.53:60781     172.16.75.116:ms-do    ESTABLISHED
  TCP    172.16.75.53:63934     lcmaaa-an-in-f3:https  TIME_WAIT
  TCP    172.16.75.53:63947     150.171.110.83:https   ESTABLISHED
  TCP    172.16.75.53:63949     172.16.75.105:ms-do    ESTABLISHED
  TCP    172.16.75.53:63951     a23-211-105-64:https   ESTABLISHED
  TCP    172.16.75.53:63952     a23-211-105-64:https   ESTABLISHED
  TCP    172.16.75.53:63958     52.123.128.14:https    ESTABLISHED
  TCP    172.16.75.53:63960     20.42.73.30:https      ESTABLISHED
  TCP    172.16.75.53:63961     20.42.73.30:https      ESTABLISHED
  TCP    172.16.75.53:63962     52.123.129.14:https    ESTABLISHED
  TCP    172.16.75.53:63963     172.16.75.116:ms-do    TIME_WAIT
  TCP    172.16.75.53:63965     172.16.75.99:ms-do     SYN_SENT
  TCP    172.16.75.53:64969     dns:https              CLOSE_WAIT
  TCP    172.16.75.53:65244     172.16.75.216:ms-do    ESTABLISHED
  TCP    172.16.75.53:65246     172.16.77.70:ms-do     ESTABLISHED
```

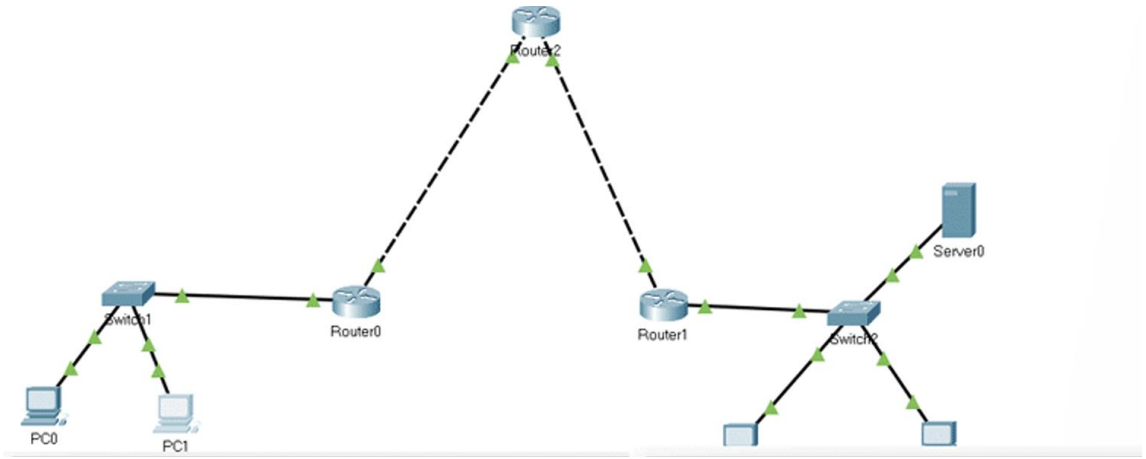

computer1
computer2
switch1
computer3
router
computer4
switch2
computer5

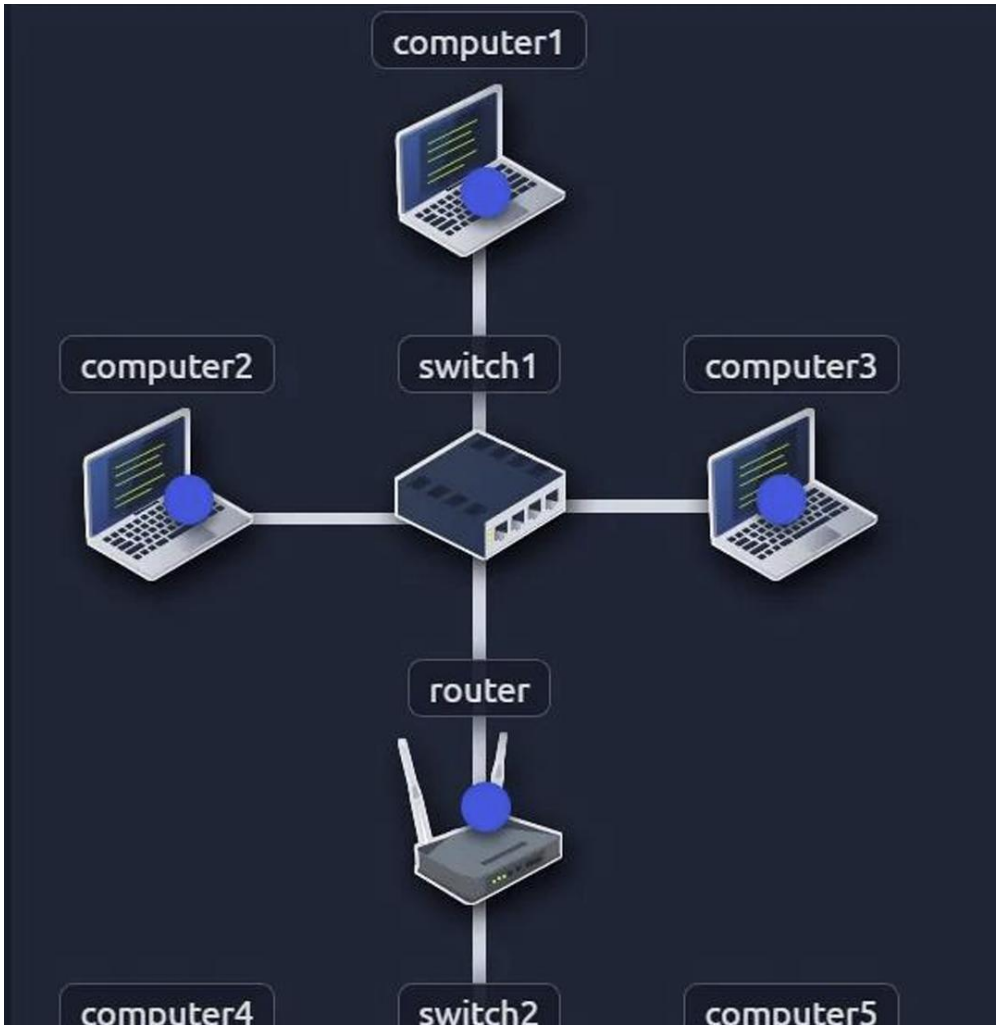

LAN 1
192.168.1.0/24
PC-PT
PC1
PC-PT
PC0
PC-PT
PC2
2960-24TT
Switch3
PC-PT
PC4
PC-PT
PC3
2901
Router0
LAN 2
192.168.2.0/24
PC-PT
PC0(1)
PC-PT
PC1(1)
PC-PT
PC2(1)
2960-24TT
Switch4
PC-PT
PC4(1)
PC-PT
PC3(1)
LAN 3
192.168.3.0/24
PC-PT
PC1(2)
PC-PT
PC0(2)
PC-PT
PC2(2)
2960-24TT
Switch1
PC-PT
PC4(2)
PC-PT
PC3(2)
2901
Router2
LAN 4
192.168.4.0/24
PC-PT
PC1(1)(1)
PC-PT
PC0(1)(1)
PC-PT
PC2(1)(1)
2960-24TT
Switch5
PC-PT
PC4(1)(1)
PC-PT
PC3(1)(1)

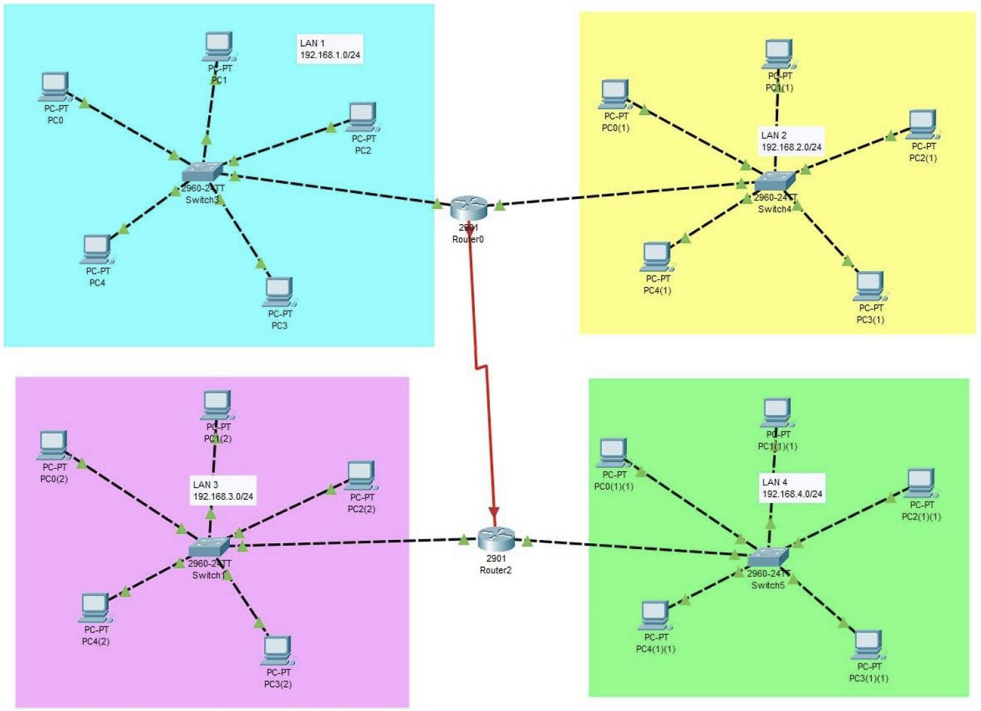

EDULA VINAY KUMAR REDDY
1Router1
PC-PT
PC0
PC-PT
PC1

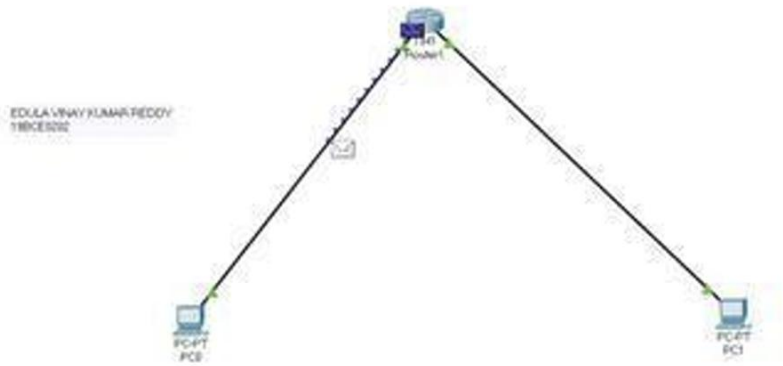

EDULA VINAY KUMAR REDDY
1Router1
PC-PT
PC0
PC-PT
PC1

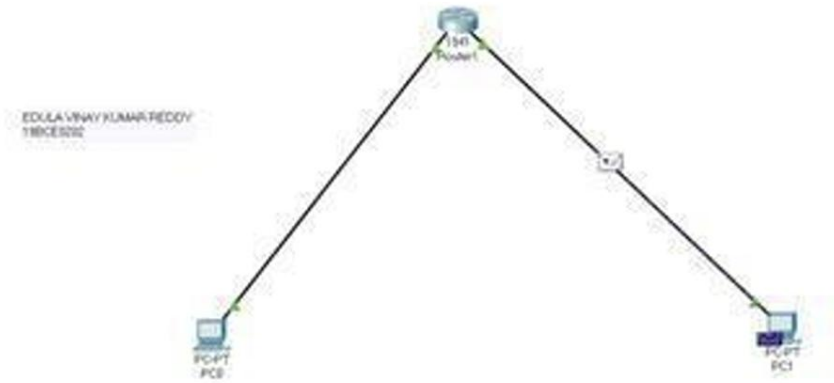

File Edit Options View Tools Extensions Help
Logical
Physical
x: 335, y: 2
[Root]
07:36:30
Wireless Router
Cable Modem
PC
Internet
Cisco.com
Laptop
Time: 01:51:04
Realtime
Simulation

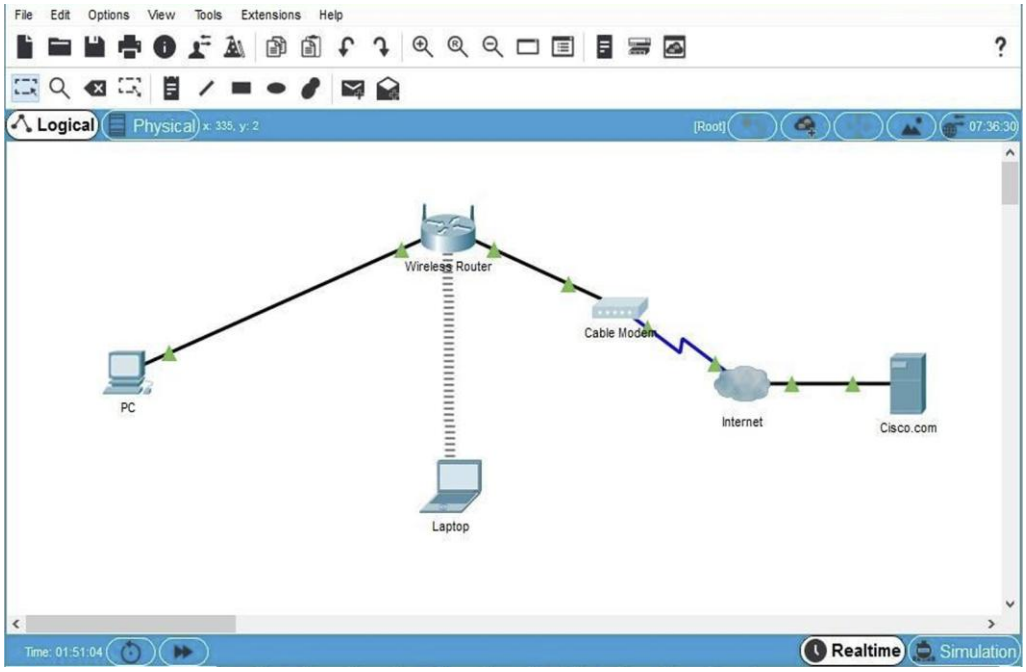

Router2
Server0
Switch1
Router0
Router1
Switch2
PC0
PC1

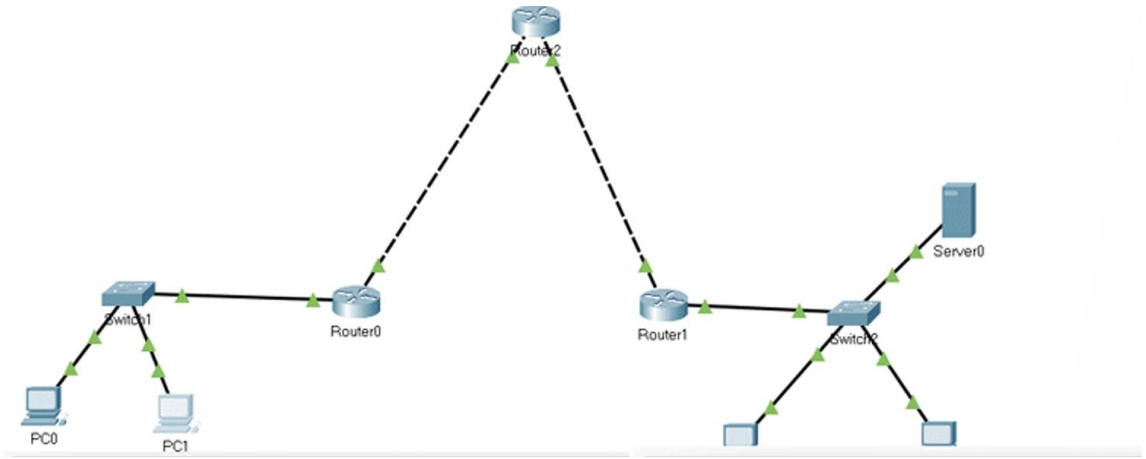

computer1
computer2
switch1
computer3
router
computer4
switch2
computer5


LAN 1
192.168.1.0/24
PC-PT
PC1
PC-PT
PC0
PC-PT
PC2
2960-24TT
Switch3
PC-PT
PC4
PC-PT
PC3
2901
Router0
LAN 2
192.168.2.0/24
PC-PT
PC0(1)
PC-PT
PC1(1)
PC-PT
PC2(1)
2960-24TT
Switch4
PC-PT
PC4(1)
PC-PT
PC3(1)
LAN 3
192.168.3.0/24
PC-PT
PC1(2)
PC-PT
PC0(2)
PC-PT
PC2(2)
2960-24TT
Switch1
PC-PT
PC4(2)
PC-PT
PC3(2)
2901
Router2
LAN 4
192.168.4.0/24
PC-PT
PC1(1)(1)
PC-PT
PC0(1)(1)
PC-PT
PC2(1)(1)
2960-24TT
Switch5
PC-PT
PC4(1)(1)
PC-PT
PC3(1)(1)


EDULA VINAY KUMAR REDDY
1Router1
PC-PT
PC0
PC-PT
PC1


EDULA VINAY KUMAR REDDY
1Router1
PC-PT
PC0
PC-PT
PC1


File Edit Options View Tools Extensions Help
Logical
Physical
x: 335, y: 2
[Root]
07:36:30
Wireless Router
Cable Modem
PC
Internet
Cisco.com
Laptop
Time: 01:51:04
Realtime
Simulation


Router2
Server0
Switch1
Router0
Router1
Switch2
PC0
PC1

Exp 12

Part A: Echo Client-Server using TCP Sockets

AIM

To write and implement an Echo Client-Server program using TCP sockets in C.

Algorithm

Server Side

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM, 0).
2. Bind the socket to an IP address and port number using bind().
3. Put the server socket in listening mode using listen().
4. Accept an incoming client connection using accept().
5. Read the message sent by the client using read() or recv().
6. Echo the same message back to the client using write() or send().
7. Close the client and server sockets using close().

Client Side

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM, 0).
2. Connect to the server's IP address and port number using connect().
3. Prompt the user for input and send the message to the server using write() or send().

4. Receive the echoed message back from the server using read() or recv().
5. Display the received message on the terminal.
6. Close the socket using close().

Code Implementation (TCP)

`tcp_server.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};

    // 1. Create socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("Socket creation failed");
```

```
        exit(EXIT_FAILURE);
    }

    // 2. Bind socket
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address,
sizeof(address)) < 0) {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }

    // 3. Listen for connections
    if (listen(server_fd, 3) < 0) {
        perror("Listen failed");
        exit(EXIT_FAILURE);
    }

    printf("[TCP Server] Waiting for incoming connections on port
%d...\n", PORT);

    // 4. Accept connection
```

```
    if ((new_socket = accept(server_fd, (struct sockaddr
*)&address, (socklen_t*)&addrlen)) < 0) {
        perror("Accept failed");
        exit(EXIT_FAILURE);
    }

    // 5 & 6. Read and Echo back
    read(new_socket, buffer, BUFFER_SIZE);
    printf("[TCP Server] Received from client: %s\n", buffer);

    send(new_socket, buffer, strlen(buffer), 0);
    printf("[TCP Server] Echoed message back to client.\n");

    // 7. Close sockets
    close(new_socket);
    close(server_fd);
    return 0;
}
```

tcp_client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
```

```
#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE];

    // 1. Create socket
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Convert IPv4 address from text to binary form
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <=
0) {
        printf("\nInvalid address / Address not supported \n");
        return -1;
```

```
    }

    // 2. Connect to server
    if (connect(sock, (struct sockaddr *)&serv_addr,
sizeof(serv_addr)) < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }

    // 3. Send message
    printf("Enter message to send to server: ");
    fgets(message, BUFFER_SIZE, stdin);
    message[strcspn(message, "\n")] = 0; // Remove newline

    send(sock, message, strlen(message), 0);
    printf("[TCP Client] Message sent.\n");

    // 4 & 5. Receive echoed message and display
    read(sock, buffer, BUFFER_SIZE);
    printf("[TCP Client] Echo received from server: %s\n", buffer);

    // 6. Close socket
    close(sock);
    return 0;
```

}

Input/Output (TCP)

Input (Client Terminal):

Enter message to send to server: Hello Networks Lab!

Output (Server Terminal):

[TCP Server] Waiting for incoming connections on port 8080...

[TCP Server] Received from client: Hello Networks Lab!

[TCP Server] Echoed message back to client.

Output (Client Terminal):

[TCP Client] Message sent.

[TCP Client] Echo received from server: Hello Networks Lab!

Part B: Echo Client-Server using UDP Sockets

AIM

To write and implement an Echo Client-Server program using UDP sockets in C.

Algorithm

Server Side

1. Create a UDP socket using socket(AF_INET, SOCK_DGRAM, 0).
2. Bind the socket to an IP address and port number using bind().
3. Receive a message and store the client's address using recvfrom().
4. Echo the message back to the client using sendto().

5. Close the socket using close().

Client Side

1. Create a UDP socket using socket(AF_INET, SOCK_DGRAM, 0).
2. Get input from the user and send the message to the server using sendto().
3. Receive the echoed message back from the server using recvfrom().
4. Display the received message on the terminal.
5. Close the socket using close().

Code Implementation (UDP)

udp_server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sockfd;
    char buffer[BUFFER_SIZE] = {0};
    struct sockaddr_in servaddr, cliaddr;
```

```
    socklen_t len = sizeof(cliaddr);

    // 1. Create socket
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&servaddr, 0, sizeof(servaddr));
    memset(&cliaddr, 0, sizeof(cliaddr));

    // 2. Bind socket
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = INADDR_ANY;
    servaddr.sin_port = htons(PORT);

    if (bind(sockfd, (const struct sockaddr *)&servaddr,
sizeof(servaddr)) < 0) {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }

    printf("[UDP Server] Waiting for data on port %d...\n", PORT);
```

```
    // 3. Receive message
    int n = recvfrom(sockfd, (char *)buffer, BUFFER_SIZE,
MSG_WAITALL, (struct sockaddr *)&cliaddr, &len);
    buffer[n] = '\0';
    printf("[UDP Server] Received from client: %s\n", buffer);

    // 4. Echo back message
    sendto(sockfd, (const char *)buffer, strlen(buffer),
MSG_CONFIRM, (const struct sockaddr *)&cliaddr, len);
    printf("[UDP Server] Echoed message back to client.\n");

    // 5. Close socket
    close(sockfd);
    return 0;
}
```

udp_client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024
```

```
int main() {
    int sockfd;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE];
    struct sockaddr_in servaddr;
    socklen_t len = sizeof(servaddr);

    // 1. Create socket
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&servaddr, 0, sizeof(servaddr));

    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    servaddr.sin_addr.s_addr = INADDR_ANY;

    // 2. Get input and send
    printf("Enter message to send to UDP server: ");
    fgets(message, BUFFER_SIZE, stdin);
    message[strcspn(message, "\n")] = 0;
```

```
    sendto(sockfd, (const char *)message, strlen(message),
MSG_CONFIRM, (const struct sockaddr *)&servaddr,
sizeof(servaddr));
    printf("[UDP Client] Message sent.\n");

    // 3 & 4. Receive echo and display
    int n = recvfrom(sockfd, (char *)buffer, BUFFER_SIZE,
MSG_WAITALL, (struct sockaddr *)&servaddr, &len);
    buffer[n] = '\0';
    printf("[UDP Client] Echo received from server: %s\n",
buffer);

    // 5. Close socket
    close(sockfd);
    return 0;
}
```

Input/Output (UDP)

Input (Client Terminal):

Enter message to send to UDP server: Testing UDP Socket

Output (Server Terminal):

[UDP Server] Waiting for data on port 8080...

[UDP Server] Received from client: Testing UDP Socket

[UDP Server] Echoed message back to client.

Output (Client Terminal):

[UDP Client] Message sent.

[UDP Client] Echo received from server: Testing UDP Socket